



Super Asgard-V

Un simulador moderno para aprender arquitectura de computadores,
directamente en el navegador

Manual de Usuario

AUTOR Jorge Márquez Torres
DIRECTOR José Sánchez Moreno

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	30/09/2026	Jorge Márquez Torres	Versión inicial del manual de usuario.

Índice general

1. Manual de instalación	1
1.1. Requisitos previos	1
1.2. Instalación nativa en Linux	1
1.2.1. Obtener el código fuente	1
1.2.2. Instalar dependencias	2
1.2.3. Ejecutar en modo desarrollo	2
1.2.4. Generar la build de producción	2
1.2.5. Ejecutar la suite de pruebas (opcional)	3
1.3. Instalación con Docker	3
1.3.1. Cómo está construida la imagen	3
1.3.2. Construir y levantar el contenedor	3
1.3.3. Otras operaciones habituales	4
1.3.4. Cambiar el puerto publicado	4
1.3.5. Solución de problemas	4
2. Manual de usuario	6
2.1. Requisitos previos	6
2.2. Interfaz de usuario	6
2.2.1. Menú principal	7
2.2.2. Menú móvil	8
2.2.3. Controles de ejecución	9
2.3. Configuración de la ejecución	10
2.3.1. Procesador	11
2.3.2. Predicción de saltos	13
2.3.3. Arquitectura vectorial	14

2.3.4.	Latencias de unidades funcionales	16
2.3.5.	Configuración específica superescalar	18
2.3.6.	Registro y visualización	21
2.4.	Cómo ejecutar programas	22
2.4.1.	Editor de código	22
2.4.2.	Banco de registros	24
2.4.3.	Memoria de datos e instrucciones	27
2.4.4.	Pipeline	27
2.4.5.	Cronograma de ejecución	28
2.4.6.	Métricas de rendimiento	29
2.4.7.	Entrada y salida por consola	32
2.4.8.	Componentes del procesador superescalar	33
2.4.9.	Registro de eventos (Log)	36
2.4.10.	Planificador estático	36
2.5.	Excepciones	37
2.5.1.	Tabla de Vectores	38
2.5.2.	Manejadores predefinidos	39
2.5.3.	Manejador por defecto	40
2.5.4.	Registro IAR	40
2.5.5.	Registro CAUSE	40
2.5.6.	Instrucción RFE	41
2.6.	Planificación estática	41
2.6.1.	Reordenamiento de instrucciones (Scheduling)	41
2.6.2.	Desenrollado de bucles (Loop unrolling)	42
2.6.3.	Renombrado estático de registros	42
2.6.4.	Relleno de slot de retardo	43
2.7.	Ejemplos de código	43

2.7.1. Cálculo del factorial de un número	43
2.7.2. Test de número primo	45
2.7.3. Algoritmo de la burbuja	46
2.7.4. Serie de Fibonacci	48
2.7.5. Cálculo del área de un círculo	48
2.7.6. Operación DAXPY vectorial	50
3. Catálogo de instrucciones	52
3.1. Aritmética y lógica entera	52
3.2. Aritmética y lógica entera con inmediatos	56
3.3. Comparaciones	61
3.4. Aritmética, comparaciones y conversiones de punto flotante	63
3.5. Acceso a memoria	67
3.6. Saltos y bifurcaciones	71
3.7. Instrucciones vectoriales	75
3.8. Directivas	78
3.9. Llamadas al sistema	81

Índice de figuras

2.1. Interfaz desarrollada para escritorio.	6
2.2. Interfaz desarrollada para dispositivos móviles.	7
2.3. Menú principal de la aplicación.	8
2.4. Menú móvil de la aplicación.	8
2.5. Controles de ejecución del simulador.	9
2.6. Controles de ejecución del simulador flotantes.	9
2.7. Parámetros de configuración del procesador.	11
2.8. Parámetros de configuración de la predicción de saltos.	13
2.9. Parámetros de configuración de la arquitectura vectorial.	14
2.10. Parámetros de configuración de latencias de unidades funcionales.	16
2.11. Parámetros de configuración del procesador superescalar.	18
2.12. Parámetros de configuración de rendimiento del sistema.	21
2.13. Editor con varias pestañas y un punto de interrupción.	23
2.14. El editor muestra sugerencias de autocompletado durante la escritura de código.	24
2.15. El editor muestra la línea con error marcada en rojo.	24
2.16. El panel de registros muestra el valor de los registros flotantes.	25
2.17. El panel de registros muestra el valor de los registros enteros.	26
2.18. El panel de registros muestra el valor de los registros vectoriales.	26
2.19. El panel de memoria resalta la última dirección accedida o modificada.	27
2.20. El pipeline con instrucciones en cada etapa durante una ejecución.	28
2.21. El pipeline superescalar con instrucciones en cada etapa durante una ejecución.	28
2.22. El cronograma muestra el historial de etapas de la ejecución.	29
2.23. Las estadísticas de la ejecución se calculan ciclo a ciclo.	30
2.24. Las estadísticas de la ejecución del superescalar muestran sus elementos propios.	32

2.25. El terminal puede pedir valores por pantalla.	33
2.26. Lista ROB con instrucciones pendientes.	34
2.27. Estación de reserva de operaciones enteras ocupada.	35
2.28. Bus común de datos con dos resultados publicados.	35
2.29. El panel de log muestra un histórico de eventos de la ejecución.	36
2.30. Diálogo con el código original a la izquierda y el optimizado con las técnicas seleccionadas a la derecha.	37

Índice de listados

1.1. Comandos para obtener el código fuente.	1
1.2. Comando para instalar las dependencias.	2
1.3. Comando para levantar el servidor de desarrollo.	2
1.4. Comando para compilar la aplicación en modo producción.	2
1.5. Comando para iniciar un servidor web.	2
1.6. Comandos para ejecutar la suite de pruebas.	3
1.7. Comandos para construir la imagen de la aplicación.	4
1.8. Comandos para realizar operaciones habituales con Docker.	4
1.9. Código modificado con otro puerto.	4
2.1. Ejemplo de reordenamiento de instrucciones para optimizar un código.	41
2.2. Ejemplo de desenrollado de bucle con factor 8.	42
2.3. Ejemplo de renombrado de registros.	42
2.4. Programa factorial.	43
2.5. Librería input.	44
2.6. Test de número primo.	45
2.7. Algoritmo de la burbuja que ordena un array y muestra el resultado por pantalla.	46
2.8. Algoritmo de Fibonacci.	48
2.9. Cálculo del área de un círculo.	48
2.10. DAXPY vectorial.	50
3.1. ADD	52
3.2. ADDU	52
3.3. SUB	53
3.4. SUBU	53
3.5. MULT	53
3.6. MULTU	54
3.7. DIV	54
3.8. DIVU	54
3.9. AND	54
3.10. OR	55
3.11. XOR	55
3.12. SLL	55
3.13. SRL	55
3.14. SRA	56
3.15. NOP	56
3.16. ADDI	56
3.17. ADDUI	57
3.18. SUBI	57
3.19. SUBUI	58
3.20. MULTI	58
3.21. DIVI	58
3.22. ANDI	59
3.23. ORI	59
3.24. XORI	59
3.25. SLLI	60
3.26. SRLI	60

3.27. SRAI	60
3.28. LHI	61
3.29. SEQ	61
3.30. SNE	61
3.31. SLT	62
3.32. SGT	62
3.33. SLE	62
3.34. SGE	62
3.35. SEQI	63
3.36. SLTI	63
3.37. ADDF	63
3.38. MULTD	64
3.39. LTF	64
3.40. LED	64
3.41. CVTF2D	65
3.42. CVTI2D	65
3.43. MOVD	66
3.44. MOVI2FP	66
3.45. MOVI2S	66
3.46. MOVF2S	67
3.47. LB	67
3.48. LBU	68
3.49. LH	68
3.50. LHU	68
3.51. LW	69
3.52. LF	69
3.53. LD	69
3.54. SB	70
3.55. SH	70
3.56. SW	70
3.57. SF	71
3.58. SD	71
3.59. BEQZ	72
3.60. BNEZ	72
3.61. BGTZ	72
3.62. BLTZ	72
3.63. BFPT	73
3.64. BFPF	73
3.65. J	73
3.66. JAL	74
3.67. JR	74
3.68. JALR	74
3.69. ADDV	75
3.70. MULTSV	75
3.71. ADDVS	75
3.72. SLTV	76
3.73. SLTSV	76
3.74. SGEVS	76

3.75. LV y SV	77
3.76. LVWS	77
3.77. LVI	77
3.78. CVI	78
3.79. CVM	78
3.80. POP	78
3.81. .data	79
3.82. .text	79
3.83. .global	79
3.84. .word	80
3.85. .byte	80
3.86. .float	80
3.87. .double	80
3.88. .space	80
3.89. .ascii	81
3.90. .asciiz	81
3.91. .align	81
3.92. IAR	82
3.93. RFE	82
3.94. TRAP 0	82
3.95. TRAP 1	82
3.96. TRAP 2	83
3.97. TRAP 3	83
3.98. TRAP 4	84
3.99. TRAP 5	84
3.100TRAP 6	85

Capítulo 1

Manual de instalación

Este capítulo describe cómo iniciar la aplicación desarrollada “Super Asgard-V”. La aplicación está disponible en <https://jmarquezt.github.io/super-asgard-v/>. Si se desea hacer una instalación local, se proporcionan dos posibilidades: instalación nativa en el servidor o equipo elegidos e instalación utilizando un contenedor Docker. Ambos caminos producen el mismo resultado, la aplicación servida a través de un servidor web y accesible por el navegador.

1.1. Requisitos previos

- **Git**, para clonar el repositorio.
- **Node.js 22** y **npm** (incluido en Node.js); necesarios para la instalación nativa.
- **Docker Engine** y el plugin **Docker Compose** (`docker compose`); necesarios para la instalación con contenedores. No hace falta tener Node.js instalado en el sistema anfitrión en este caso: la imagen de Docker proporcionada ya lo incluye.
- Un navegador web moderno (Chrome, Firefox, Edge) con JavaScript habilitado.

1.2. Instalación nativa en Linux

Se exponen los pasos necesarios para realizar la instalación en un sistema Linux o en Windows usando el subsistema de Windows para Linux (WSL).

1.2.1. Obtener el código fuente

El Listado 1.1 presenta los comandos necesarios para descargar el código fuente de la aplicación en la carpeta deseada.

```
1 cd /carpeta-deseada
2 git clone https://github.com/jmarquezt/super-asgard-v.git super-
  asgard-v
3 cd super-asgard-v
```

Listado 1.1: Comandos para obtener el código fuente.

1.2.2. Instalar dependencias

Se recomienda utilizar `npm ci` en lugar de `npm install`: instala exactamente las versiones fijadas en `package-lock.json`, garantizando una instalación reproducible (es el mismo comando que usan el despliegue automático con GitHub Actions y el desarrollo). El Listado 1.2 presenta el comando necesario para descargar e instalar las dependencias del proyecto.

```
1 npm ci
```

Listado 1.2: Comando para instalar las dependencias.

1.2.3. Ejecutar en modo desarrollo

El comando `npm start` levanta un servidor de desarrollo con recarga automática al modificar el código fuente. El Listado 1.3 presenta el comando necesario para ejecutar la aplicación en modo desarrollo.

```
1 npm start
```

Listado 1.3: Comando para levantar el servidor de desarrollo.

La aplicación es accesible desde el navegador en <http://localhost:4200>.

1.2.4. Generar la build de producción

```
1 npm run build
```

Listado 1.4: Comando para compilar la aplicación en modo producción.

El comando del Listado 1.4 genera los ficheros estáticos compilados y optimizados en `dist/super-asgard-v/browser/`. Al tratarse de una aplicación estática (sin backend), cualquier servidor web puede publicarlos; por ejemplo, `http-server` (paquete de npm, se instala una sola vez con `npm install -g http-server`) o servidores como Nginx o Apache:

```
1 http-server dist/super-asgard-v/browser -p 8080
```

Listado 1.5: Comando para iniciar un servidor web.

Usando `http-server` como se muestra en el Listado 1.5, la aplicación es accesible desde el navegador en <http://localhost:8080>.

1.2.5. Ejecutar la suite de pruebas (opcional)

El Listado 1.6 presenta los comandos que hay que ejecutar para lanzar la suite de pruebas deseada.

```
1  npm test                # suite completa
2  npm run coverage        # suite completa + informe de cobertura
3  npm run test:unit       # solo pruebas unitarias
4  npm run test:integration # solo pruebas de integración de
    componentes
5  npm run test:functional  # solo pruebas funcionales (programas ASG
    completos)
```

Listado 1.6: Comandos para ejecutar la suite de pruebas.

1.3. Instalación con Docker

El repositorio incluye un `Dockerfile` multi-etapa y un `docker-compose.yml` en su raíz, pensados para desplegar la aplicación sin necesidad de tener instaladas las dependencias en la máquina anfitriona.

1.3.1. Cómo está construida la imagen

El `Dockerfile` define dos etapas:

1. **Compilación:** la imagen (`node:22-alpine`) copia el código fuente, ejecuta `npm ci` y compila la aplicación en modo producción con `ng build -configuration production`, generando `dist/super-asgard-v/browser`.
2. **Servidor:** la imagen (`nginx:alpine`) copia únicamente los ficheros estáticos generados en la etapa anterior a `/usr/share/nginx/html` y expone el puerto 80. La imagen final no contiene Node.js ni el código fuente, solo un servidor Nginx sirviendo los ficheros compilados.

El fichero `docker-compose.yml` define un único servicio (`super-asgard-v`) que arranca esta última imagen y mapea el puerto 80 del contenedor al puerto **4815** de la máquina anfitriona, con reinicio automático (`restart: always`).

1.3.2. Construir y levantar el contenedor

Como `docker-compose.yml` referencia la imagen por nombre (`super-asgard-v:latest`) sin una sección `build:`, hay que construirla explícitamente con `docker build` antes de levantar el servicio la primera vez (y cada vez que el código fuente cambie) tal y como se muestra en el Listado 1.7.

```
1 # 1. Construir la imagen a partir del Dockerfile del repositorio
2 docker build -t super-asgard-v:latest .
3
4 # 2. Levantar el contenedor en segundo plano
5 docker compose up -d
```

Listado 1.7: Comandos para construir la imagen de la aplicación.

La aplicación queda disponible en <http://localhost:4815>.

1.3.3. Otras operaciones habituales

El Listado 1.8 presenta una serie de comandos habituales para trabajar con Docker.

```
1 docker compose logs -f      # ver los logs de Nginx en tiempo real
2 docker compose down        # parar y eliminar el contenedor
3 docker compose up -d --force-recreate  # reiniciar tras reconstruir
   la imagen
```

Listado 1.8: Comandos para realizar operaciones habituales con Docker.

1.3.4. Cambiar el puerto publicado

Si el puerto 4815 ya está en uso en la máquina anfitriona, basta con editar el mapeo de puertos en `docker-compose.yml` (el número de la izquierda es el del anfitrión; el de la derecha, 80, es interno del contenedor y no debe cambiarse) como se muestra en el Listado 1.9.

```
1 ports:
2 - "8080:80"    # la aplicación pasaría a estar en http://localhost
   :8080
```

Listado 1.9: Código modificado con otro puerto.

1.3.5. Solución de problemas

- **npm ci falla con un error de versión de Node:** comprobar con `node -v` que la versión instalada es 22.x. Herramientas como `nvm` permiten tener varias versiones de Node conviviendo en el mismo sistema.
- **El puerto 4200, 4815 u 8080 ya está en uso:** liberar el puerto (`lsof -i :4200` en Linux para ver qué proceso lo ocupa) o publicar la aplicación en otro puerto siguiendo las instrucciones de la sección anterior.

- **Docker no encuentra la imagen `super-asgard-v:latest`:** falta ejecutar `docker build` antes de `docker compose up`: `docker-compose.yml` no construye la imagen automáticamente, solo la referencia por nombre.
- **Cambios en el código no se reflejan tras reconstruir la imagen:** Docker puede reutilizar capas en caché; para forzar una reconstrucción completa se puede utilizar `docker build --no-cache -t super-asgard-v:latest .`

Capítulo 2

Manual de usuario

Este capítulo sirve como punto de entrada y manual de referencia a alumnos y docentes que quieran utilizar la aplicación.

2.1. Requisitos previos

Es necesario disponer de acceso a la aplicación instalada ya sea en un entorno local o cloud como se explicó en el Capítulo 1, además de un navegador web para acceder a la misma.

2.2. Interfaz de usuario

La interfaz se presenta de dos posibles maneras, según se acceda utilizando un dispositivo de escritorio o un dispositivo móvil. Se considera un dispositivo móvil aquel con una resolución de pantalla inferior a 600px y orientación vertical o con una resolución de pantalla inferior a 960px y orientación horizontal.

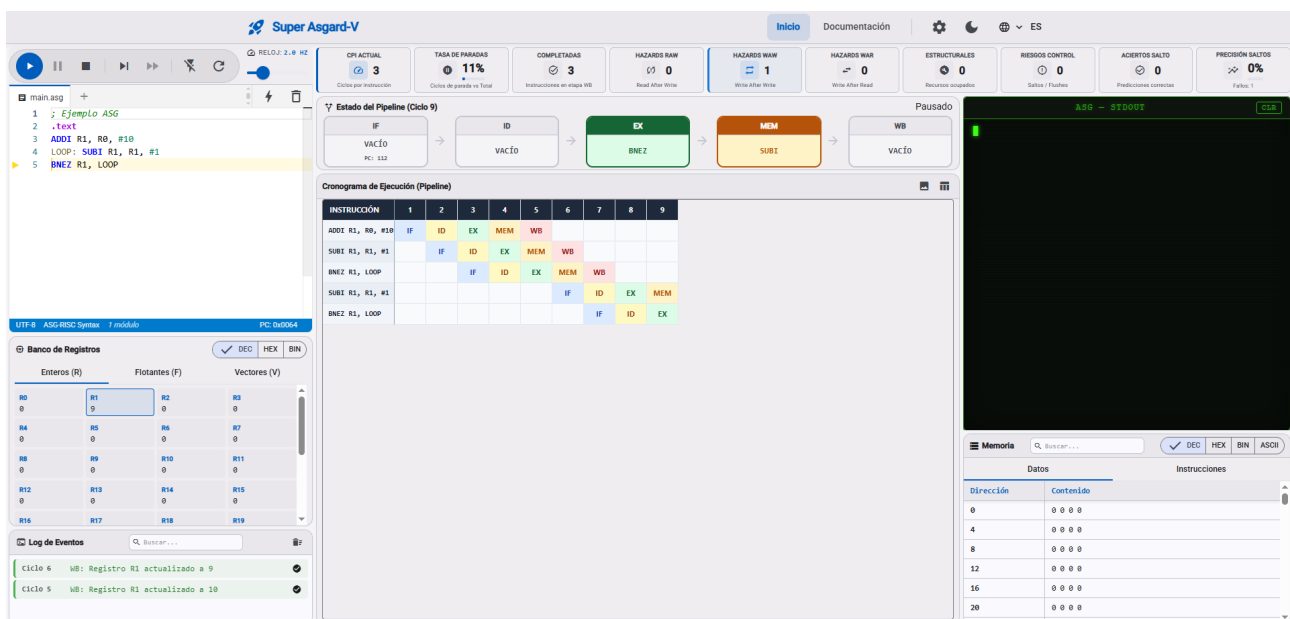


Figura 2.1: Interfaz desarrollada para escritorio.

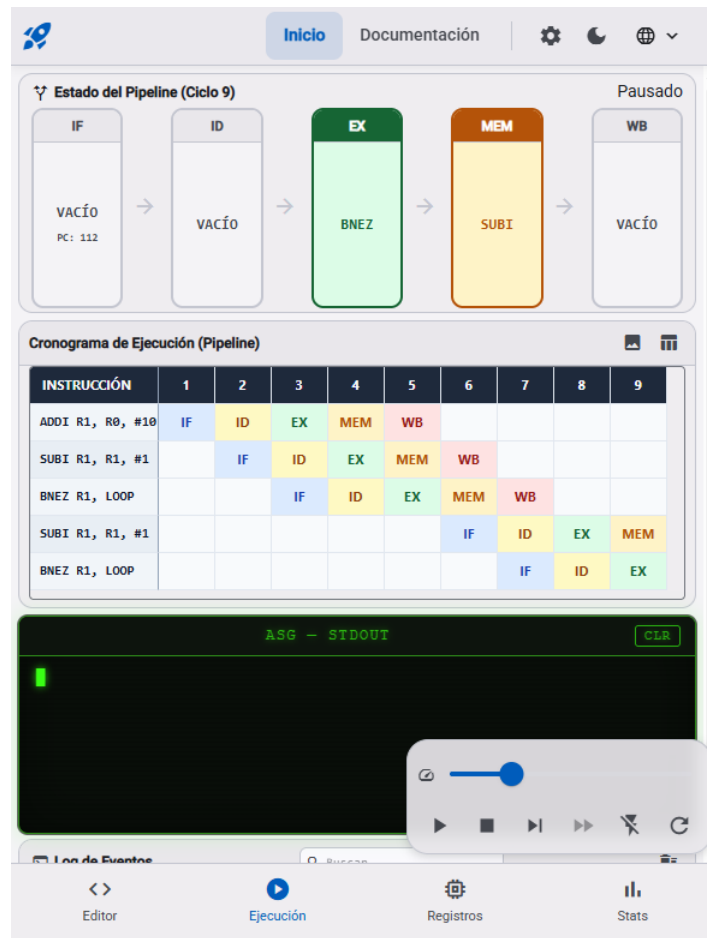


Figura 2.2: Interfaz desarrollada para dispositivos móviles.

La Figura 2.1 muestra cómo es la interfaz para un dispositivo de escritorio, dividida en tres columnas, poniendo el foco en el pipeline y el cronograma de la ejecución (en la columna central), rodeando a estos del resto de componentes del procesador, así el editor del código fuente y los registros quedan en la columna izquierda y la consola de entrada / salida y la memoria en la columna derecha. Por otro lado, la Figura 2.2 muestra cómo en dispositivos móviles con la pantalla más pequeña, la interfaz se divide en pestañas agrupando los diferentes componentes por etapa dentro del proceso, pero manteniendo los controles de ejecución siempre visibles.

2.2.1. Menú principal

El menú principal se muestra siempre visible en la parte de arriba de la pantalla, incluye el logo y nombre de la aplicación junto a las opciones básicas de navegación. La Figura 2.3 muestra las opciones del menú principal de la aplicación que, de izquierda a derecha son:

1. **Inicio:** carga la pantalla por defecto de la aplicación, el simulador.
2. **Documentación:** carga una pantalla especial que recoge tanto las instrucciones y directivas disponibles en la arquitectura como un pequeño manual de usuario de la aplicación.

3. **Ajustes:** presenta un formulario en popup con las opciones de configuración del simulador.
4. **Tema:** permite intercambiar entre un tema oscuro y claro de la interfaz.
5. **Idioma:** consiste en un selector que permite cambiar el idioma de la aplicación. Los idiomas disponibles a fecha de la redacción de este manual son “Inglés” y “Español”.

Hay que aclarar que todos los textos del apartado “Documentación” así como las traducciones del idioma “Inglés” han sido generados con la Inteligencia Artificial Generativa de Google “Gemini” y revisados posteriormente antes de su inclusión.

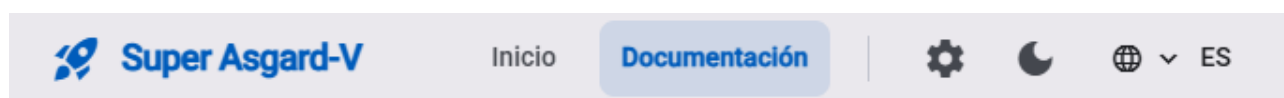


Figura 2.3: Menú principal de la aplicación.

2.2.2. Menú móvil

El menú de pestañas que aparece en dispositivos móviles se muestra en la parte de abajo de la pantalla, siguiendo las líneas de diseño actuales tanto de iOS como de Android. La Figura 2.4 muestra las opciones del menú móvil de la aplicación que, de izquierda a derecha son:

1. **Editor:** es la pantalla por defecto en móvil y punto de entrada de la aplicación. Muestra el editor a pantalla completa.
2. **Ejecución:** muestra los componentes de pipeline, cronograma, consola de E/S y el log de eventos.
3. **Registros:** muestra los componentes de registros y memorias, además de los específicos del superescalar.
4. **Stats:** muestra las estadísticas de la ejecución.

El menú principal sigue estando visible siempre en la parte de arriba para poder seleccionar las opciones de navegación principales de la aplicación.

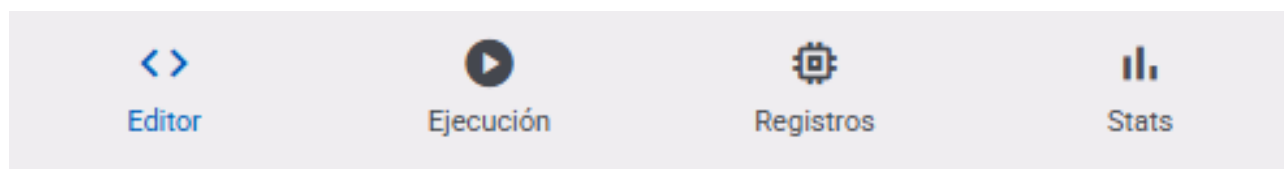


Figura 2.4: Menú móvil de la aplicación.

2.2.3. Controles de ejecución

Tanto en la interfaz de escritorio como en la interfaz móvil los controles de la ejecución están siempre visibles y a disposición del usuario para controlar el sistema. La Figura 2.5 muestra cómo los controles de ejecución en la interfaz de escritorio están integrados en el editor, mientras que la Figura 2.6 muestra cómo aparecen flotantes sobre el menú móvil, apareciendo siempre independientemente de la pestaña seleccionada.

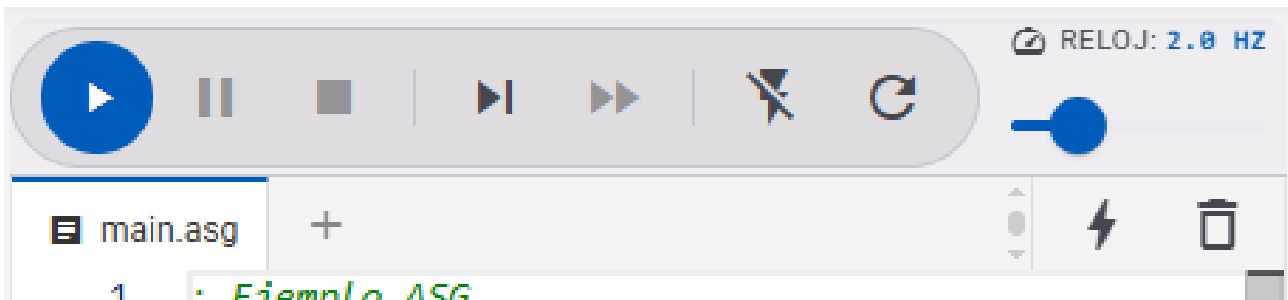


Figura 2.5: Controles de ejecución del simulador.

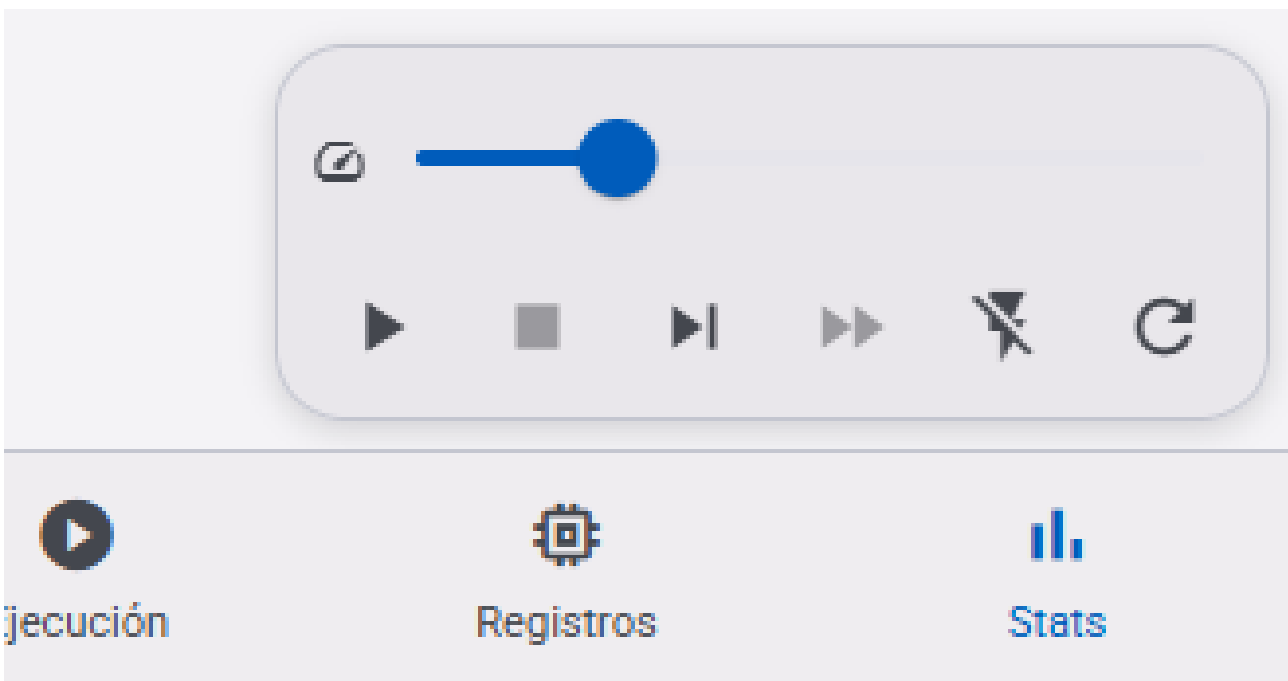


Figura 2.6: Controles de ejecución del simulador flotantes.

Los controles disponibles para el control de la ejecución son los siguientes:

1. **Ejecutar:** compila y ejecuta el código disponible en el editor o continúa una ejecución en pausa hasta el final del mismo sin interrupciones (salvo que encuentre una excepción).
2. **Pausar:** pausa la ejecución actual manteniendo el estado de ejecución, pudiendo reanudar la misma desde la instrucción en que se pausó.

3. **Detener ejecución:** para la ejecución actual sin mantener el estado de ejecución, siendo necesario reiniciar la ejecución desde la primera instrucción.
4. **Ciclo único (step):** en una ejecución pausada, ejecuta el siguiente ciclo al actual, pausando el procesador al término del mismo.
5. **Ejecutar hasta breakpoint:** ejecuta el código disponible en el editor o continúa una ejecución en pausa hasta el siguiente breakpoint encontrado.
6. **Modo rendimiento:** permite configurar el simulador para ejecutar 500 ciclos de reloj por ciclo real, aumentando el rendimiento.
7. **Reiniciar procesador:** para el procesador y reinicia el estado de ejecución al inicial.
8. **Selector de velocidad:** permite seleccionar la velocidad (ciclos por segundo) a la que se ejecutan los ciclos para aumentar o disminuir el rendimiento de la máquina.

2.3. Configuración de la ejecución

Desde el menú principal (icono engranaje - settings) se accede a la vista con el formulario para configurar los parámetros de la máquina que se quiere simular. Estos parámetros están agrupados según la categoría a la que aplican y se pueden cambiar individualmente, además se proporciona una configuración “de fábrica” con la que operar.

2.3.1. Procesador

Procesador

Tipo de procesador

Segmentado (Pipelined)

Tamaño de memoria (KB)*

16384

Configura el espacio total de direccionamiento.

Adelantamiento de datos

Slot de retardo de salto

La instrucción posterior a cada salto se ejecuta siempre antes de que el salto sea efectivo.

Figura 2.7: Parámetros de configuración del procesador.

La Figura 2.7 muestra los parámetros de configuración disponibles para la categoría “procesador”, esta categoría permite personalizar los siguientes parámetros.

Tipo de procesador

Permite seleccionar el tipo de procesador de entre los desarrollados que ejecutará el programa simulado.

Procesador monociclo (“No segmentado (Ciclo único)” en la interfaz): ejecuta una instrucción completa antes de comenzar la siguiente. Cada instrucción pasa por las etapas (IF, ID, EX, MEM, WB) de forma secuencial, cada una dura entre uno y varios ciclos de reloj según la latencia de la instrucción. No presenta riesgos de datos ni estructurales, ya que cada instrucción termina antes de que la siguiente empiece. El CPI es variable según la instrucción (dependiendo de la latencia) y no dispone de adelantamiento de datos ni predicción de saltos. Es útil para el aprendizaje inicial y para comparar el rendimiento con los procesadores segmentados.

Procesador segmentado: ejecuta las instrucciones de manera solapada en las etapas (IF, ID, EX, MEM, WB), permitiendo hasta 5 instrucciones simultáneas. Dispone de adelantamiento de datos para resolver dependencias RAW sin detener la segmentación y predicción de saltos para continuar la ejecución antes de la resolución del mismo. Ambas técnicas permiten mejorar

la velocidad de ejecución y reducir detenciones. El objetivo es alcanzar un CPI de 1, donde se completa una instrucción en cada ciclo de reloj.

Procesador superescalar: arquitectura con ejecución fuera de orden que utiliza el algoritmo de Tomasulo (planificación dinámica) para maximizar el paralelismo y aumentar el rendimiento. El Reorder Buffer (ROB) mantiene el orden del programa. Las estaciones de reserva RS y las unidades FU se encargan de la ejecución fuera de orden y permiten procesar varias instrucciones en el mismo ciclo. Este procesador puede alcanzar un CPI menor que 1, pero a costa de aumentar el hardware del equipo.

Tamaño de memoria

Tamaño del espacio de memoria de datos en bytes (por defecto 4096 bytes / 4 KB). Aumentar este valor permite programas más grandes, pero incrementa el uso de memoria del simulador. La memoria de instrucciones es independiente y no se puede configurar. El valor mínimo es 1024 y el máximo 65536. Los valores introducidos deben ser potencia de 2.

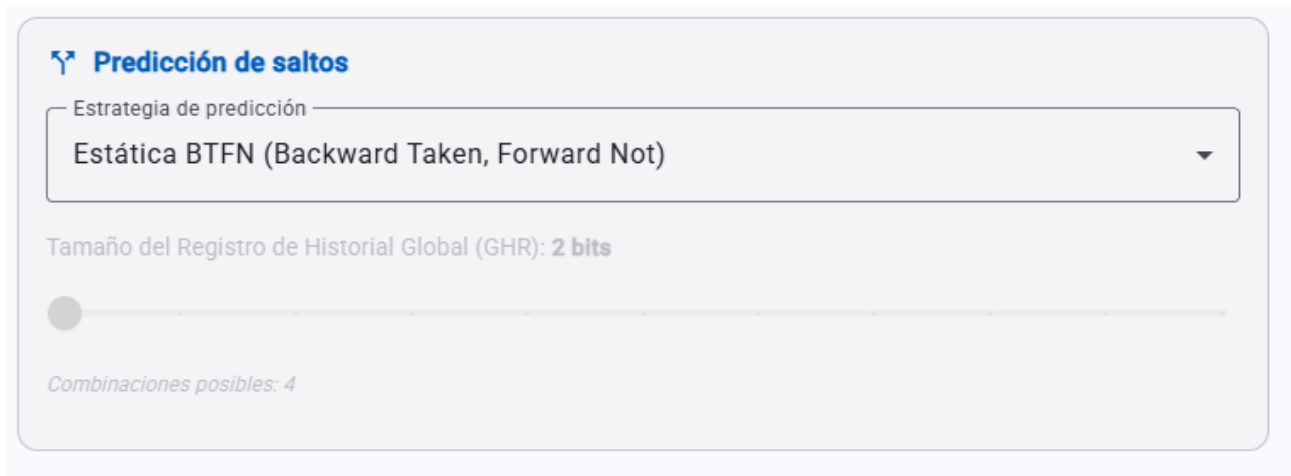
Adelantamiento de datos

Activa o desactiva el envío de datos entre etapas del pipeline antes de su consolidación en los registros o memoria (adelantamiento). Con el adelantamiento activo, los resultados disponibles en las etapas EX/MEM y MEM/WB se propagan directamente a la entrada de la etapa EX de la instrucción siguiente, de forma que la única dependencia RAW que provoca una parada es la de una instrucción que usa el dato cargado por un load inmediatamente anterior (load-use). Al desactivarlo, la instrucción dependiente debe esperar en ID a que el dato esté disponible en la etapa WB de la instrucción anterior, aumentando el CPI. Valor por defecto: activado. Solo disponible si en tipo de procesador se selecciona “segmentado”. El procesador monociclo por sus características de construcción no dispone de adelantamiento de datos y en el superescalar es obligatorio por sus características.

Branch Delay Slot

Opción “Slot de retardo de salto” en la interfaz. Permite configurar si la instrucción posterior a un salto se debe ejecutar siempre, se tome o no el salto. Valor por defecto: desactivado. Solo disponible si en tipo de procesador se selecciona “segmentado”. El procesador monociclo no dispone de slot de retardo, puesto que ejecuta una instrucción completa antes de la siguiente, y en el superescalar está desactivado por sus características especulativas, ya que se ejecutan instrucciones siguientes al salto antes de su resolución.

2.3.2. Predicción de saltos



Predicción de saltos

Estrategia de predicción

Estática BTFN (Backward Taken, Forward Not)

Tamaño del Registro de Historial Global (GHR): 2 bits

Combinaciones posibles: 4

Figura 2.8: Parámetros de configuración de la predicción de saltos.

La Figura 2.8 muestra los parámetros de configuración disponibles para la categoría “predicción de saltos”, esta categoría permite personalizar los siguientes parámetros. Solo está disponible si en tipo de procesador se selecciona “segmentado” o “superescalar”: el procesador monociclo no dispone de predicción de saltos.

Estrategia de predicción

Permite seleccionar entre una de las ocho estrategias de predicción de saltos disponibles en el sistema:

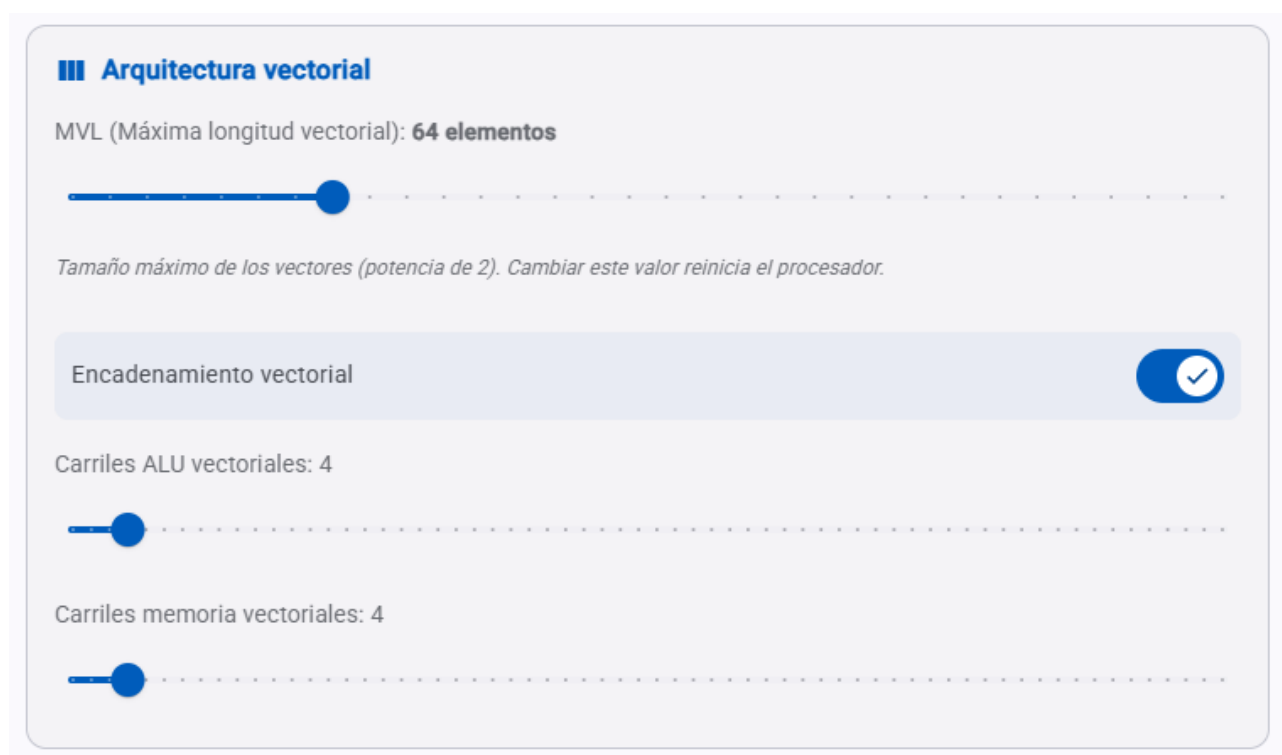
- **Ninguna:** sin predictor; se continúa siempre por la ruta secuencial, es decir, se predice el salto como no tomado (“Ninguna (Siempre No Tomado)” en la interfaz).
- **Estática siempre tomado:** se predice el salto siempre como tomado.
- **Estática BTFN:** Backward Taken, Forward Not. Se predice como tomado si el salto es hacia atrás en el código y como no tomado si el salto es hacia adelante.
- **Dinámica 1-bit:** predictor de 1 bit por entrada, recuerda el último resultado.
- **Dinámica 2-bit:** predictor de Smith con contador saturante de 2 bits.
- **Dinámica 2-bit gshare:** predictor con historial global combinado aplicando XOR con la dirección de salto.
- **Híbrida por torneo:** combina un predictor local con un predictor gshare global y evalúa el mejor resultado.
- **RAS:** pila de direcciones de retorno para predecir el destino de instrucciones JR, siendo su principal uso predecir el retorno de las subrutinas.

Valor seleccionado por defecto: Ninguna.

Tamaño del registro GHR

Permite configurar los bits del registro de historial global. Mínimo 2 bits y máximo 12 bits. Valor por defecto: 2 bits. Cuanto mayor sea el registro, mayor será el número de combinaciones posibles y patrones más largos, pero puede llevar a un menor porcentaje de acierto. Solo disponible si en estrategia de predicción se selecciona “Dinámica 2-bit gshare” o “Híbrida por torneo”.

2.3.3. Arquitectura vectorial



Arquitectura vectorial

MVL (Máxima longitud vectorial): **64 elementos**

Tamaño máximo de los vectores (potencia de 2). Cambiar este valor reinicia el procesador.

Encadenamiento vectorial ☒

Carriles ALU vectoriales: 4

Carriles memoria vectoriales: 4

Figura 2.9: Parámetros de configuración de la arquitectura vectorial.

La Figura 2.9 muestra los parámetros de configuración disponibles para la categoría “arquitectura vectorial”, esta categoría permite personalizar los siguientes parámetros.

Máxima longitud vectorial MVL

Número máximo de elementos que puede contener cada registro vectorial. Determina cuántas iteraciones de un bucle vectorizado caben en una sola instrucción vectorial. Los bucles con más elementos que el MVL necesitan dividirse en varios tramos del tamaño del MVL. El registro VLR nunca puede ser mayor que el valor de MVL configurado. Siempre en potencias de 2, con valor mínimo de 8 y valor máximo de 256. Por defecto con valor 64.

Encadenamiento vectorial (chaining)

Permite que una instrucción vectorial que depende del resultado de otra (dependencia RAW) empiece a consumir sus elementos a medida que se van produciendo, sin esperar a que el vector completo esté calculado. Al desactivarlo, la instrucción dependiente espera a que la anterior obtenga el resultado completo, lo que puede aumentar el tiempo de ejecución vectorial en varios ciclos. Valor por defecto: activado. Solo disponible si en tipo de procesador se selecciona “segmentado” o “superescalar”. El procesador monociclo por sus características de construcción no dispone de encadenamiento.

Carriles ALU vectoriales

Permite ajustar el número de “lanes” disponibles al realizar operaciones ALU vectoriales. Cada carril procesa un elemento por ciclo. El valor mínimo es 1 y el valor máximo 64. El valor por defecto es 4.

Carriles memoria vectoriales

Permite ajustar el número de “lanes” disponibles al realizar operaciones de memoria vectorial. Cada carril procesa un elemento por ciclo. El valor mínimo es 1 y el valor máximo 64. El valor por defecto es 4.

Figura 2.10: Parámetros de configuración de latencias de unidades funcionales.

16

operación permanece en la etapa EX según la operación que realiza (el valor configurable está siempre entre 1 y 256 ciclos). Valores altos de latencia simulan un hardware más lento para esa operación mientras que valores más bajos simulan un hardware más rápido.

1. **Operaciones enteras:** multiplicación entera por defecto a 7 y la división entera por defecto a 24.
2. **Operaciones de punto flotante:**
 - Suma / resta FP simple: por defecto a 2.
 - Multiplicación FP simple: por defecto a 7.
 - División FP simple: por defecto a 24.
 - Suma / resta FP doble: por defecto a 4.
 - Multiplicación FP doble: por defecto a 14.
 - División FP doble: por defecto a 48.
3. **Operaciones vectoriales:** se corresponden con la latencia inicial de la unidad funcional antes de poder operar (en el caso de las operaciones de almacenamiento se refiere a la latencia final hasta que el dato está disponible).
 - Memoria vectorial: por defecto a 12.
 - Suma vectorial: por defecto a 6.
 - Multiplicación vectorial: por defecto a 7.
 - División vectorial: por defecto a 20.
 - Comparación vectorial: por defecto a 1.

2.3.5. Configuración específica superescalar

 **Configuración Superescalar**

Mínima

Equilibrada

Agresiva

Tipo de Estaciones de Reserva
Distribuidas

Ancho de emisión (Issue)
4 instr/ciclo

Organización de las estaciones de reserva

Tamaño del ROB*
64

Ancho del CDB*
4

Entradas del Reorder Buffer (16-256)

Ancho del Common Data Bus (1-16)

RS por unidad
4

Unidades Funcionales

ALUs Enteras 2	Mult. Enteros 1	Div. Enteros 1
Sumadores FP 1	Mult. FP 1	Div. FP 1
Unidades Carga/Alm. 2	Unidades de Salto 1	Unidades Mem Vect 1
ALUs Vectoriales 1	Mult. Vectoriales 1	Div. Vectoriales 1

Solapado de inicio vectorial



Permite que una instrucción vectorial comience su fase de inicio mientras otra aún procesa en la misma UF

Figura 2.11: Parámetros de configuración del procesador superescalar.

La Figura 2.11 muestra los parámetros de configuración disponibles para la categoría “superescalar”, esta categoría permite personalizar las opciones de ejecución para el procesador superescalar, solo aplican si el tipo de procesador seleccionado es “superescalar”.

Tipo y tamaño de las estaciones de reserva

Permite seleccionar cómo se organizarán las estaciones de reserva entre “distribuidas” (cada unidad funcional tiene su propia RS; permite configurar el número de estaciones de reserva por unidad funcional, por defecto 4), “centralizadas” (una única RS compartida por todas las unidades funcionales. Permite configurar el tamaño de la estación, por defecto 32) y “por clústeres” (las unidades funcionales se agrupan en 4 clústeres según el tipo de operación. Cada uno tiene su propio tamaño de RS configurable).

Ancho de emisión

Permite configurar cuántas instrucciones simultáneas se pueden enviar a las estaciones de reserva en un ciclo de reloj. También es el número de instrucciones que se capturan de manera simultánea en la etapa IF. Los valores disponibles son 1, 2, 4 y 8. Por defecto 4.

Tamaño del ROB

Permite configurar cuántas entradas simultáneas en espera puede tener el buffer de reordenamiento, es decir, cuántas instrucciones pueden estar “en vuelo” a la vez. Si el ROB se llena, el procesador deja de capturar nuevas instrucciones en la etapa IF hasta que las existentes vayan saliendo haciendo commit en la etapa RI. Puede ser un valor entre 16 y 256. Valor por defecto 64.

Ancho del CDB

Permite configurar el número de resultados que se pueden publicar en el bus común de datos en un ciclo de reloj. Ampliar el ancho del CDB permite reducir cuellos de botella cuando varias unidades funcionales terminan en el mismo ciclo. Puede ser un valor entre 1 y 16. Por defecto 4.

Unidades funcionales

Permite configurar el número de unidades funcionales de cada tipo que hay disponibles en el procesador para realizar las operaciones. Aumentar el número de unidades de un tipo es de ayuda si el programa tiene varias instrucciones de ese tipo independientes entre sí esperando para ejecutarse. El número de unidades de cada tipo puede estar entre 1 y 8.

1. ALU enteras: por defecto a 2.

2. Multiplicación entera: por defecto a 1.
3. División entera: por defecto a 1.
4. Suma / resta FP: por defecto a 1.
5. Multiplicación FP: por defecto a 1.
6. División FP: por defecto a 1.
7. Unidades carga / almacenamiento: por defecto a 2.
8. Unidades de salto: por defecto a 1.
9. Unidades de memoria vectorial: por defecto a 1.
10. ALU vectoriales: por defecto a 1.
11. Multiplicación vectorial: por defecto a 1.
12. División vectorial: por defecto a 1.

Solapamiento de inicio vectorial

Opción “Solapado de inicio vectorial” en la interfaz. Permite que una instrucción vectorial comience su fase de inicialización en una unidad funcional que sigue ocupada por otra instrucción vectorial anterior terminando de procesar sus elementos, reduciendo el tiempo total de procesamiento entre instrucciones vectoriales. Valor por defecto: activado.

Presets de la configuración

Se han preparado tres configuraciones predefinidas jugando con los parámetros del superescalar anteriores para un ajuste automático:

- **Mínima:** Ancho de emisión = 2, ROB = 32, RS centralizada de 16, 1 ALU entera, 1 unidad de memoria, CDB = 2. Simula un procesador superescalar con configuración de hardware mínima.
- **Equilibrada** (valor por defecto): Ancho de emisión = 4, ROB = 64, RS distribuida (4 por unidad), 2 ALUs enteras, 2 unidades de memoria, CDB = 4. Simula un procesador superescalar de rendimiento medio.
- **Agresiva:** Ancho de emisión = 8, ROB = 128, RS agrupada en clústeres grandes, 4 ALUs enteras, 3 unidades de memoria, CDB = 8. Simula un procesador superescalar de alto rendimiento.

2.3.6. Registro y visualización

Figura 2.12: Parámetros de configuración de rendimiento del sistema.

La Figura 2.12 muestra los parámetros de configuración disponibles para la categoría “registro y visualización”, esta categoría permite personalizar dos opciones para mejorar el rendimiento general del sistema:

1. **Registro de eventos:** permite activar o desactivar completamente el componente de logs para no degradar el rendimiento con registros de log de la ejecución. Valor por defecto: activado.
2. **Nivel de detalle:** si la opción anterior está habilitada, permite restringir qué tipos de mensajes de log mostrar:
 - **Debug:** muestra todos los logs de la aplicación. Perfecto para hacer debug de un programa pero peor en rendimiento en programas grandes.
 - **Relevante** (valor por defecto): muestra solo los mensajes de éxito y los errores. Modo balanceado.
 - **Warning:** muestra solo mensajes de aviso y de error.
 - **Error:** muestra solo mensajes de error.
3. **Modo del cronograma:** define cómo debe mostrarse el cronograma de la ejecución. “En vivo” (valor por defecto) actualiza en tiempo real. “Al finalizar” genera el cronograma completo al terminar la ejecución del programa, lo que mejora el rendimiento. “Desactivado” no genera ningún cronograma.

2.4. Cómo ejecutar programas

No es necesario un conocimiento previo de la herramienta para comenzar a ejecutar programas. Los pasos principales son:

1. **Escribe el código:** en el editor, por defecto ya se dispone de una pestaña abierta, se escribe o se pega un código ASG.
2. **Pulsa ejecutar:** se pulsa el botón de reproducir, en los controles de ejecución. Al pulsarlo la aplicación compila el código introducido a instrucciones que el procesador simulado puede entender y empieza a ejecutarlas automáticamente, una tras otra, hasta el final.
3. **Observa los resultados:** durante la ejecución del programa, los paneles de registros, memoria, pipeline y cronograma muestran cómo cambian los valores cuando la instrucción que los modifica se ejecuta. También se puede seguir la ejecución en el editor, cuando una instrucción pasa a ejecutarse se remarca la línea actual en el editor.
4. **Si algo falla:** si el código contiene un error, la aplicación no ejecuta nada, se resalta en rojo la línea que contiene el error en el editor y se muestra un mensaje en el panel de “Log de eventos”. Si el error se produce durante la ejecución, se ejecutará el manejador de errores correspondiente y la salida del programa variará dependiendo del contenido de este último.
5. **Para volver a empezar desde cero:** se pulsa el botón de reiniciar procesador, en los controles de ejecución. Esta acción restaura la máquina a su estado inicial y la ejecución del programa a la primera instrucción.

Una vez en ejecución el programa, la herramienta dispone de varios paneles para hacer el seguimiento de los resultados.

2.4.1. Editor de código

El editor integrado soporta varias pestañas simultáneas, cada una con su propio módulo de código fuente independiente. La Figura 2.13 muestra el editor de la aplicación con varias pestañas abiertas y las diferentes opciones disponibles.

- **Añadir módulo:** el botón “+” crea un módulo adicional vacío. Al ejecutar un programa, todos los módulos abiertos se ensamblan juntos como si fueran ficheros de un mismo programa, compartiendo etiquetas y memoria de datos.
- **Cerrar pestaña:** el botón “x” elimina el módulo de forma permanente. Esta acción no se puede revertir.
- **Limpiar pestaña:** borra todo el contenido de la pestaña activa sin cerrarla.
- **Planificador estático:** abre el planificador estático y carga el código de la pestaña activa.

- **Puntos de interrupción:** permite añadir puntos de interrupción en las líneas deseadas, para ello basta con hacer clic en el margen izquierdo de la línea deseada, lo que hará que aparezca un punto rojo, señal de que se ha añadido correctamente.
 - Para eliminar un punto de interrupción, basta con pulsar sobre el punto rojo y desaparecerá.
 - Al pulsar en ejecutar, la ejecución avanzará hasta que la instrucción en esa línea vaya a entrar al pipeline, momento en que se detendrá, permitiendo inspeccionar el estado de la máquina en ese punto.
 - Los puntos de interrupción solo pueden colocarse sobre líneas que contengan una instrucción real (no en comentarios, directivas o líneas vacías).
- **Barra de estado inferior:** muestra el número de módulos abiertos y el valor actual del contador de programa (PC) en hexadecimal.

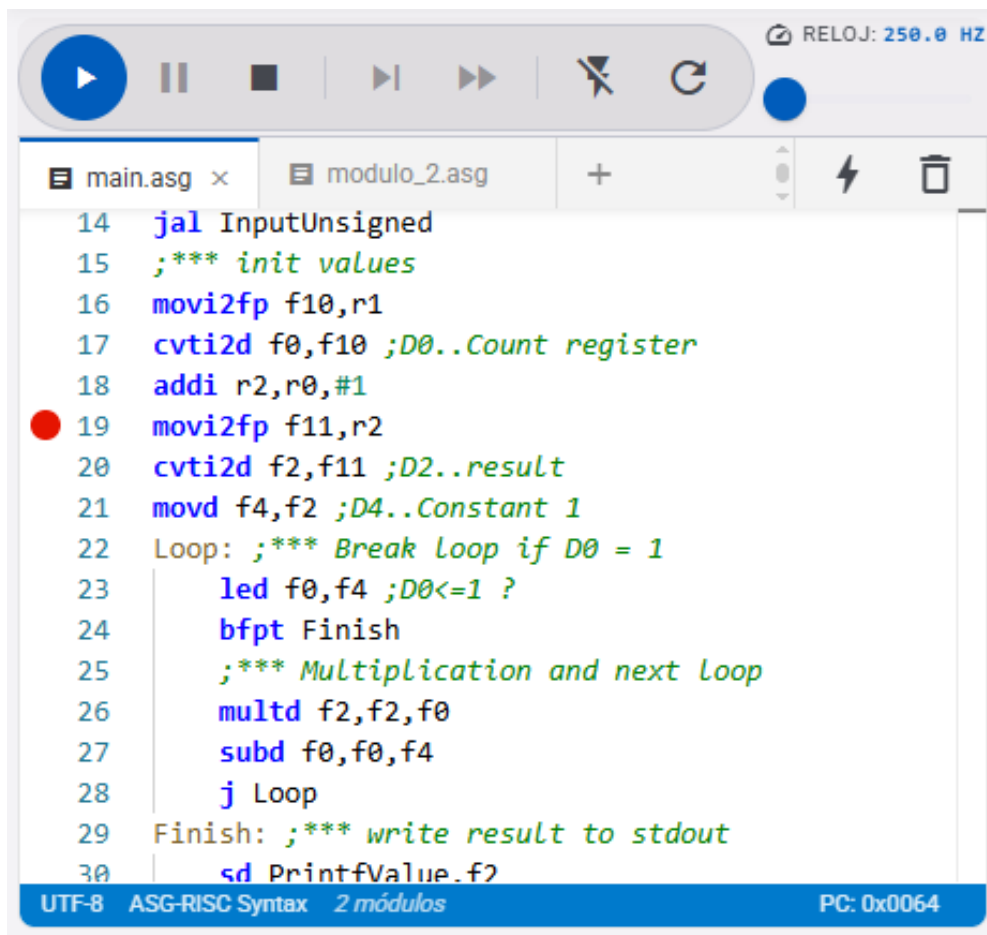


Figura 2.13: Editor con varias pestañas y un punto de interrupción.

Durante el proceso de escritura del código, el editor va sugiriendo el autocompletado de palabras clave, registros y directivas. La Figura 2.14 presenta cómo el editor muestra sugerencias durante la escritura del código. El código escrito no se ve plano, el editor dota a las palabras clave, registros, directivas y comentarios de diferentes colores para resaltarlos.

Si el código proporcionado contiene errores de ensamblado, la pestaña y línea exactas donde está el error se activan y resaltan en rojo automáticamente. La Figura 2.15 presenta cómo el editor muestra el error en una línea del código.

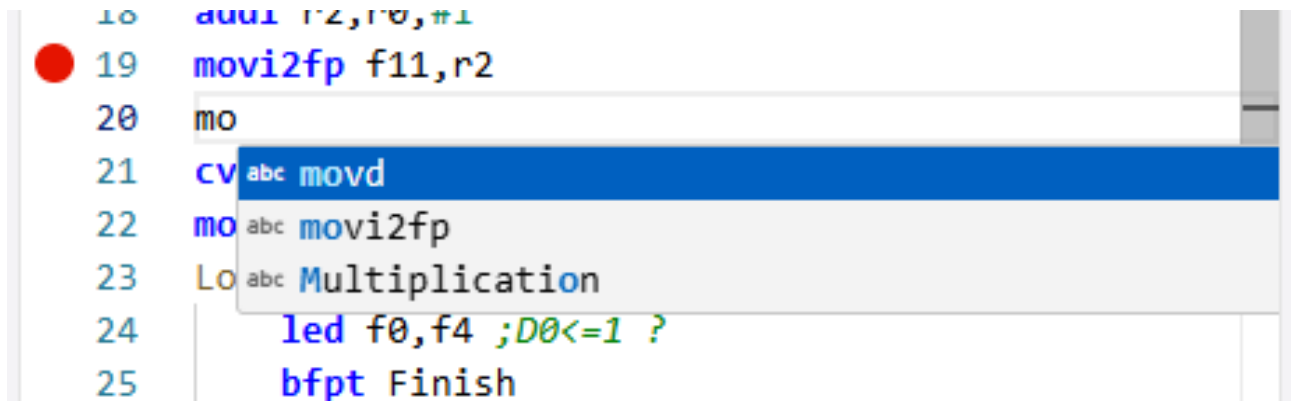


Figura 2.14: El editor muestra sugerencias de autocompletado durante la escritura de código.

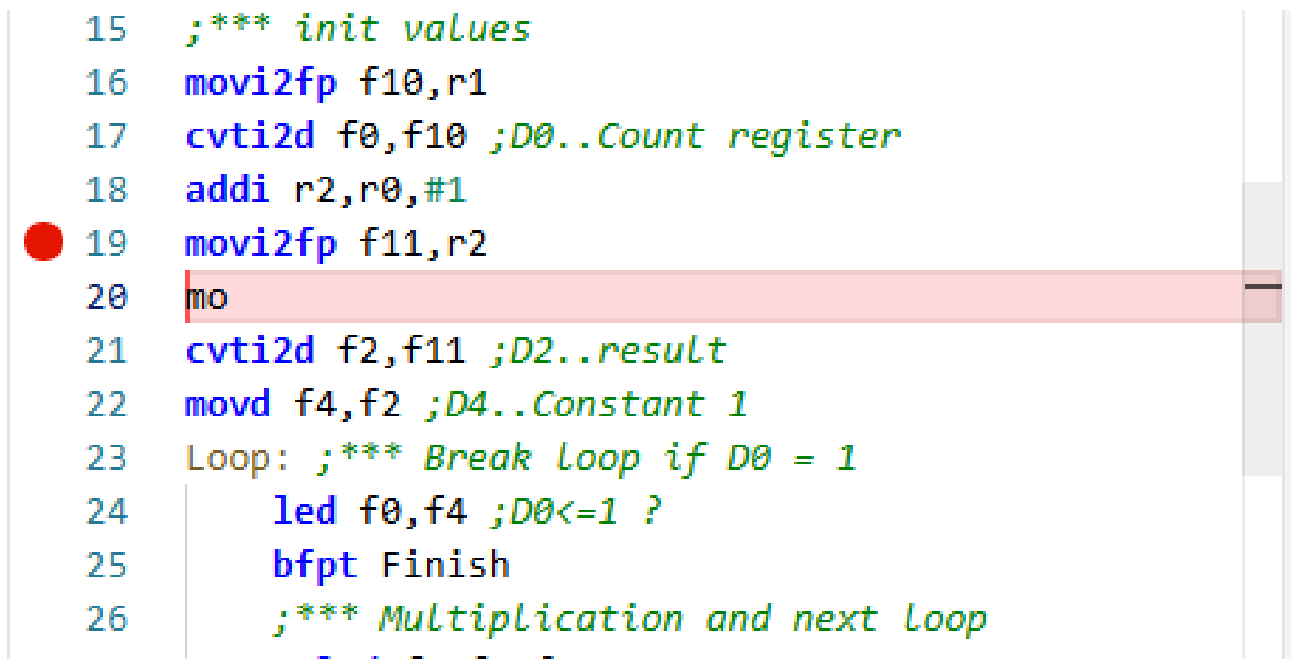


Figura 2.15: El editor muestra la línea con error marcada en rojo.

2.4.2. Banco de registros

El panel del banco de registros dispone de tres pestañas:

- **Enteros (R):** muestra los registros R0-R31 además del registro especial IAR (dirección de retorno de interrupción). R0 siempre vale 0 y no puede modificarse. La Figura 2.17 muestra cómo el panel presenta los registros enteros.

- Cuando el valor de uno de los registros se modifica en un ciclo, se muestra una animación y se resalta visualmente en color para una mejor identificación. En el modo superescalar, el registro pendiente de recibir su valor real desde el buffer de reordenamiento se marca como ocupado y muestra su robTag asociado de renombramiento. El selector DEC/HEX/BIN de la cabecera cambia el formato de todos los valores numéricos del panel.

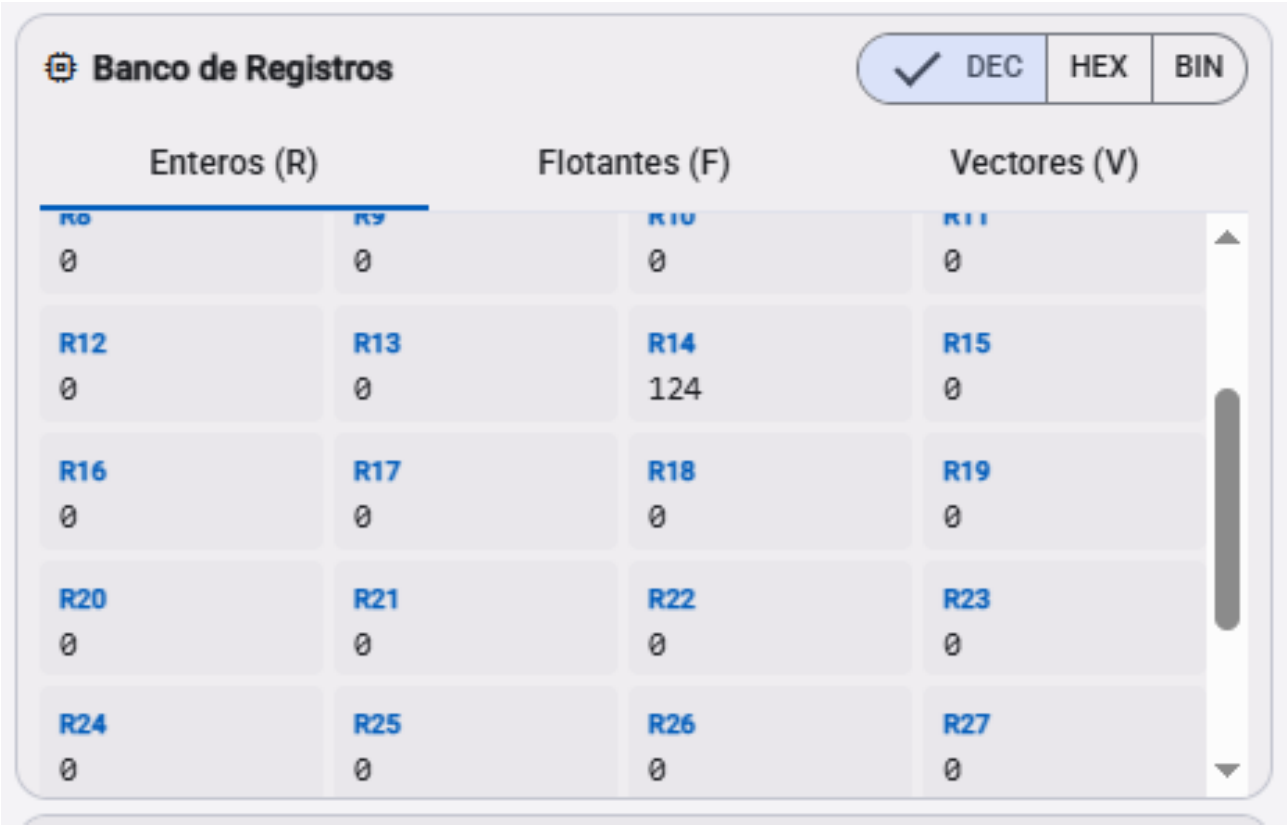


Figura 2.17: El panel de registros muestra el valor de los registros enteros.



Figura 2.18: El panel de registros muestra el valor de los registros vectoriales.

2.4.3. Memoria de datos e instrucciones

El panel de vista de memoria dispone de dos pestañas análogas que muestran el contenido de la memoria de datos y el de la memoria de instrucciones como una tabla de dos columnas con los valores de la dirección y el contenido de esta.

Al igual que en el panel de registros, es posible modificar el formato de los valores del panel con el selector DEC/HEX/BIN/ASCII de la cabecera del panel. Este componente también incluye un buscador en la cabecera que filtra las filas de la tabla mostrada por dirección o por contenido. La fila correspondiente a la última dirección leída o escrita se resalta, lo que permite comprobar visualmente que las instrucciones del código están accediendo a la posición esperada. La Figura 2.19 presenta cómo se visualiza la memoria de la máquina en la aplicación.



Memoria	
Buscar...	
☒ DEC ☐ HEX ☐ BIN ☐ ASCII	
Datos	Instrucciones
Dirección	Contenido
276	97 100 111 32
280	70 105 110 97
284	108 105 122 97
288	100 111 46 10
292	0 0 1 0
296	0 0 0 0

Figura 2.19: El panel de memoria resalta la última dirección accedida o modificada.

2.4.4. Pipeline

Durante una ejecución, el panel de pipeline resume con colores en qué etapa interna se encuentra cada instrucción en el ciclo actual. Si la etapa se encuentra vacía, aparecerá vacía y si se encuentra bloqueada aparecerá en rojo y con la palabra “stall”. La Figura 2.20 presenta un pipeline segmentado durante la ejecución de un programa.



Figura 2.20: El pipeline con instrucciones en cada etapa durante una ejecución.

Para el procesador superescalar, se ha modificado el componente visual adaptándolo a un pipeline de seis etapas: IF (captura), ID (decodificación), II (emisión a las estaciones de reserva), EX (ejecución), WR (escritura del resultado en el CDB) y RI (retirada o commit desde el ROB). La Figura 2.21 muestra el pipeline del procesador superescalar.



Figura 2.21: El pipeline superescalar con instrucciones en cada etapa durante una ejecución.

2.4.5. Cronograma de ejecución

El panel del cronograma permite visualizar un diagrama de Gantt con la ejecución del programa. Cada fila muestra una instrucción del programa y las columnas muestran los ciclos de ejecución, mostrando, para cada instrucción, las etapas por las que ha pasado dicha instrucción durante la ejecución y en qué ciclos ha ocurrido. Cada etapa se muestra con un color diferente. Las instrucciones descartadas (que no debieron ejecutarse por una mala predicción de un salto) se muestran tachadas y con opacidad. La Figura 2.22 presenta la visualización del cronograma de una ejecución.

El panel permite exportar el cronograma de la ejecución en formato CSV o en PNG con los botones disponibles en la parte superior derecha del componente. El modo de renderizado del cronograma (en vivo, al finalizar, o desactivado) se controla desde la configuración. En programas muy largos, el modo “al finalizar” o “desactivado” mejoran el rendimiento del navegador.

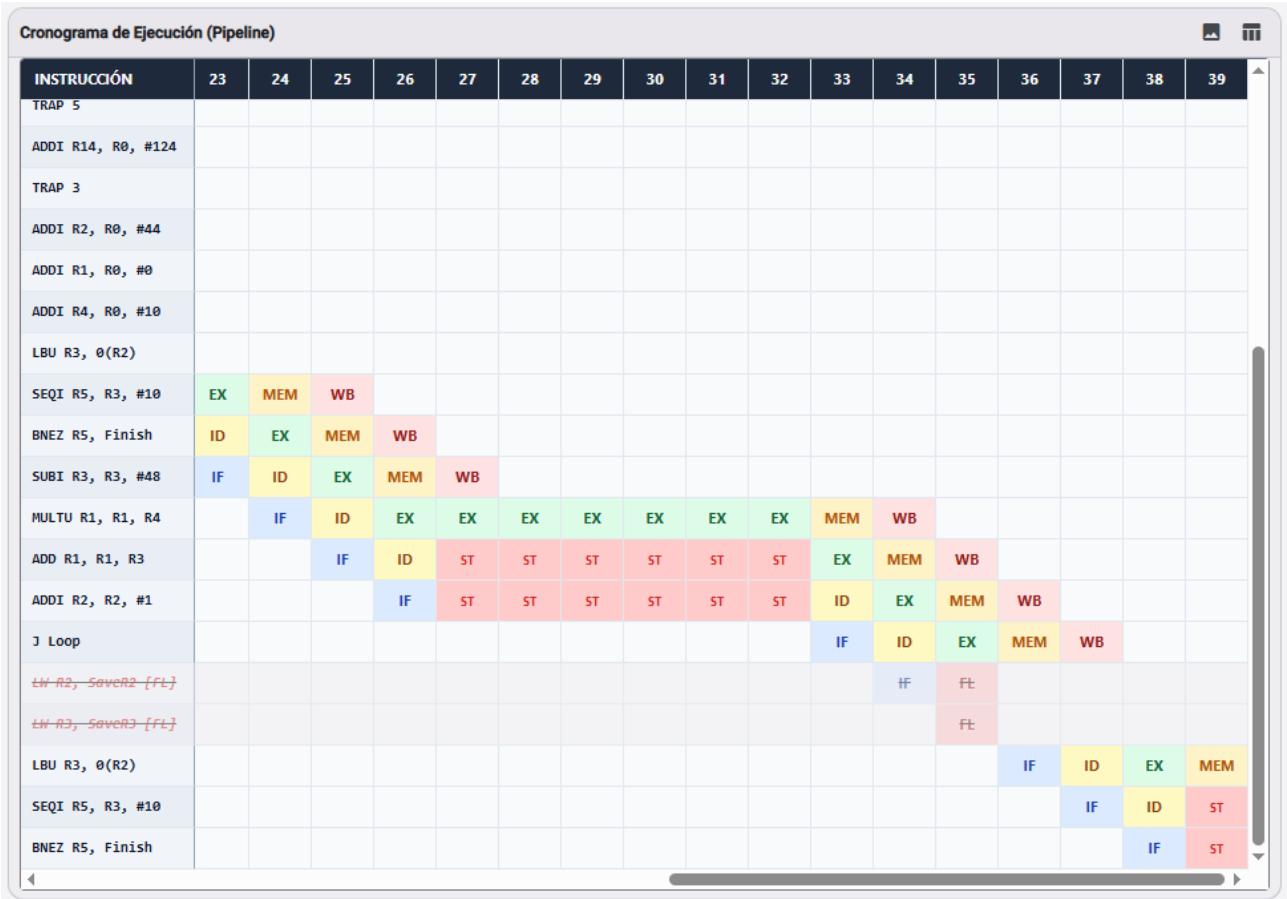


Figura 2.22: El cronograma muestra el historial de etapas de la ejecución.

2.4.6. Métricas de rendimiento

Este panel muestra diversas métricas de rendimiento y estadísticas que se calculan automáticamente cada ciclo de ejecución. La Figura 2.23 muestra las estadísticas de ejecución en un procesador segmentado. Las estadísticas mostradas son:

- **CPI (Ciclos por instrucción):** cuántos ciclos de reloj de media hacen falta para completar cada instrucción. Cuanto más bajo sea este valor, mejor. El valor ideal es 1.0. Valores mayores indican que el procesador tuvo que detenerse por algún riesgo.
- **Tasa de paradas:** porcentaje de ciclos que el procesador estuvo detenido en lugar de procesando instrucciones. Esta estadística se desglosa en varias: RAW, WAR, WAW, Estructurales (recursos ocupados) y de Control (saltos mal predichos y vaciado del pipeline).
- **Instrucciones completadas:** número de instrucciones que completan todas las etapas de la segmentación.
- **Precisión de saltos:** porcentaje de aciertos del predictor de saltos. Esta estadística solo se calcula si el programa incluye saltos condicionales. Se muestra también el número de predicciones correctas.



Figura 2.23: Las estadísticas de la ejecución se calculan ciclo a ciclo.

El panel es ligeramente diferente si se selecciona el procesador “superescalar”. La Figura 2.24 muestra cómo las estadísticas incluyen el uso de las estructuras del procesador. Las estadísticas mostradas son:

- **IPC (Instrucciones por ciclo)**: es la medida inversa al CPI. Mide cuántas instrucciones se pueden completar en un ciclo de reloj. Cuanto mayor sea el valor, mayor rendimiento.

- **Ciclos:** número de ciclos totales de ejecución del programa.
- **Instrucciones completadas:** número de instrucciones que completan todas las etapas de la segmentación.
- **Precisión de saltos:** porcentaje de aciertos del predictor de saltos. Esta estadística solo se calcula si el programa incluye saltos condicionales. Se muestra también el número de predicciones correctas y el número de fallos.
- **Vaciados de buffer:** número de veces que el buffer de lectura ha tenido que ser vaciado por un fallo en la predicción de un salto (perdiendo rendimiento al descartar esas instrucciones en vuelo).
- **Uso del ROB:** porcentaje de uso en el ciclo actual del buffer de reordenamiento. Útil para saber la ocupación.
- **Uso de las RS:** porcentaje de uso en el ciclo actual de las estaciones de reserva. Útil para saber la ocupación.
- **Paradas de captura de instrucciones:** número de veces que el procesador ha tenido que parar de leer nuevas instrucciones porque el buffer de lectura está lleno.
- **Paradas de entrada al ROB:** número de veces que el procesador ha tenido que parar de enviar nuevas entradas al ROB porque está lleno. Un valor alto indica que hay partes del programa que saturan el procesador y que conviene aumentar el tamaño del ROB.
- **Paradas de entrada a RS:** número de veces que el procesador ha tenido que parar de enviar nuevas entradas a las estaciones de reserva porque están llenas. Un valor alto indica que hay partes del programa que saturan el procesador y que conviene aumentar el tamaño de las estaciones de reserva.
- **Paradas estructurales:** número de veces que una instrucción lista para ejecutarse ha tenido que esperar porque todas las unidades funcionales de su tipo estaban ocupadas. Un valor alto indica que conviene aumentar el número de unidades funcionales de ese tipo.



Figura 2.24: Las estadísticas de la ejecución del superescalar muestran sus elementos propios.

2.4.7. Entrada y salida por consola

Los programas pueden interactuar con el usuario mediante llamadas al sistema. Para ello, se dispone de un “terminal” en el que se imprime por pantalla la salida de los programas y en el que se reciben los datos por parte del usuario. Cuando un programa requiere que el usuario introduzca datos, se detiene la ejecución y el cursor del terminal queda parpadeando a la espera

del dato. El usuario debe introducir los datos posicionándose en el terminal y pulsar “enter” cuando haya terminado, lo que hará que se reanude la ejecución. La Figura 2.25 muestra cómo la aplicación pide al usuario que introduzca un dato a través del terminal. El terminal conserva el historial de la ejecución y puede vaciarse con el botón “CLR” de la cabecera.

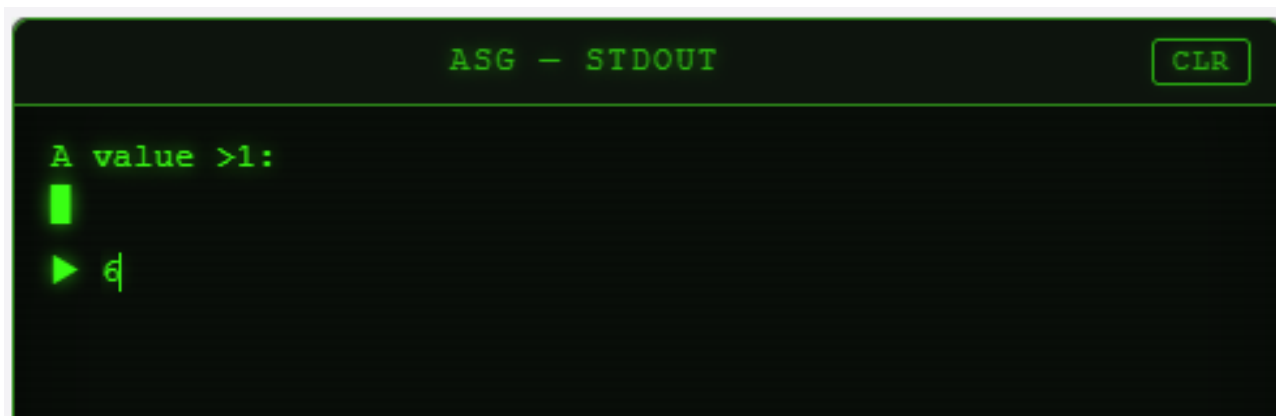


Figura 2.25: El terminal puede pedir valores por pantalla.

2.4.8. Componentes del procesador superescalar

El procesador superescalar dispone de tres componentes extra para procesar las instrucciones. La aplicación dispone de un panel visual para cada uno de ellos.

Reorder buffer (ROB)

Es una lista circular que contiene cada una de las instrucciones del programa en ejecución que han entrado al procesador en orden de entrada. Garantiza que los resultados de la ejecución fuera de orden se retiren en orden en la etapa RI. La Figura 2.26 presenta la visualización del ROB de la aplicación. La columna estado indica si está finalizada (y cuando llegue a la cabeza del ROB podrá retirarse), emitida (a su estación de reserva correspondiente) o en espera (de ser emitida). El panel también muestra el opcode que originó la entrada, así como su registro destino (si lo tuviera) y su valor a escribir. El panel también muestra en su cabecera la ocupación como (Nº de entradas ocupadas / Nº de entradas totales).

Reorder Buffer (ROB)					Ocupación: 15 / 64
Tag	Estado	Opcode	Dest	Valor	PC
7	-	-	-	-	0x0000
8	F	BNEZ	-	0	0x0000
9	F	SUBI	R1	5	0x0000
10	E	BNEZ	-	-	0x0000
11	E	SUBI	R1	-	0x0000

Figura 2.26: Lista ROB con instrucciones pendientes.

Estaciones de reserva (RS)

Las instrucciones se emiten a su estación de reserva correspondiente para ser ejecutadas y esperan aquí a que sus operandos estén listos. Si alguno de sus operandos no está resuelto, se muestra una etiqueta con la fila del ROB que lo generará. Cada estación de reserva muestra su ocupación en la cabecera como (Nº entradas ocupadas / Nº de entradas totales). Y por cada fila muestra el opcode, el valor de sus operandos (si están resueltos se mostrará el valor y si no el robTag que lo resolverá), el destino de la operación (robTag que actualizará con su resultado) y el estado en que se encuentra. La Figura 2.27 muestra la ocupación de una estación de reserva de operaciones con enteros ocupada durante una ejecución.



INT_ALU				Ocupación: 4 / 4
OP	V1/Q1	V2/Q2	DEST	ESTADO
SUBI	ROB[13]	1	ROB[15]	Espera
SUBI	5	1	ROB[11]	Espera
SUBI	ROB[15]	1	ROB[17]	Espera
SUBI	ROB[11]	1	ROB[13]	Espera

Figura 2.27: Estación de reserva de operaciones enteras ocupada.

Common Data Bus (CDB)

Cuando una unidad funcional termina de calcular un resultado, lo publica en este bus para que las estaciones de reserva que lo esperan actualicen sus operandos en el mismo ciclo. El panel visual muestra el tipo de la unidad funcional que ha publicado el resultado, el ciclo en el que lo ha publicado, el robTag que actualiza y el valor del resultado. La Figura 2.28 presenta un CDB con dos resultados publicados en el ciclo 22.



BRANCH	Ciclo 18
Tag:	ROB[8]
Valor:	0
INT_ALU	Ciclo 18
Tag:	ROB[9]
Valor:	5

Figura 2.28: Bus común de datos con dos resultados publicados.

2.4.9. Registro de eventos (Log)

Durante la ejecución de programas, la herramienta dispone de un sistema de depuración para que el usuario pueda hacer un seguimiento más detallado de la misma. El panel muestra un histórico de todos los eventos relevantes que ocurren durante la ejecución: instrucciones capturadas, finalizadas, detenciones detectadas, saltos tomados o no tomados, fallos de predicción, excepciones o errores de ensamblado. Cada entrada muestra el ciclo en el que ocurrió, además de un fondo de color y un icono según su severidad (información, aviso, error o éxito). Al igual que la memoria, dispone de un buscador que filtra las entradas por el contenido de su mensaje. Permite también limpiar el panel sin afectar al resto de componentes. El registro se reinicia automáticamente al reiniciar el procesador.

Es el primer panel a consultar cuando un programa no se comporta como se espera, ya que suele señalar la causa. La Figura 2.29 muestra el panel con eventos durante una ejecución.

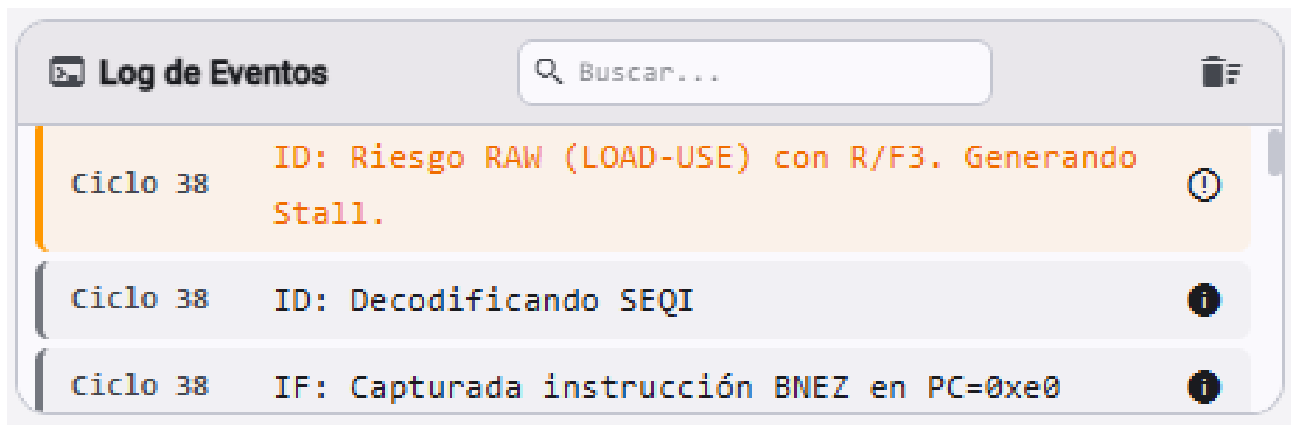


Figura 2.29: El panel de log muestra un histórico de eventos de la ejecución.

2.4.10. Planificador estático

El botón de optimización disponible en el editor abre el diálogo “Planificador Estático (Optimización)”, que permite aplicar técnicas para optimizar el código actual del editor. El código no se modifica hasta que el usuario acepta los cambios propuestos por el planificador. La Figura 2.30 presenta el planificador estático con las diferentes opciones disponibles. Estas opciones son:

- **Reordenamiento (scheduling):** reordena instrucciones independientes para reducir las paradas por dependencias.
- **Desenrollado de bucles (Loop unrolling):** desenrolla bucles un número configurable de veces (factor entre 2 y 8, por defecto 2) para reducir saltos y aumentar el paralelismo entre iteraciones.
- **Renombrado estático de registros:** elimina falsas dependencias (WAW/WAR) asignando registros temporales distintos.
- **Relleno de slot de retardo (Delayed branch):** rellena el slot de retardo tras un salto con una instrucción útil que se debe ejecutar independientemente del resultado del salto.

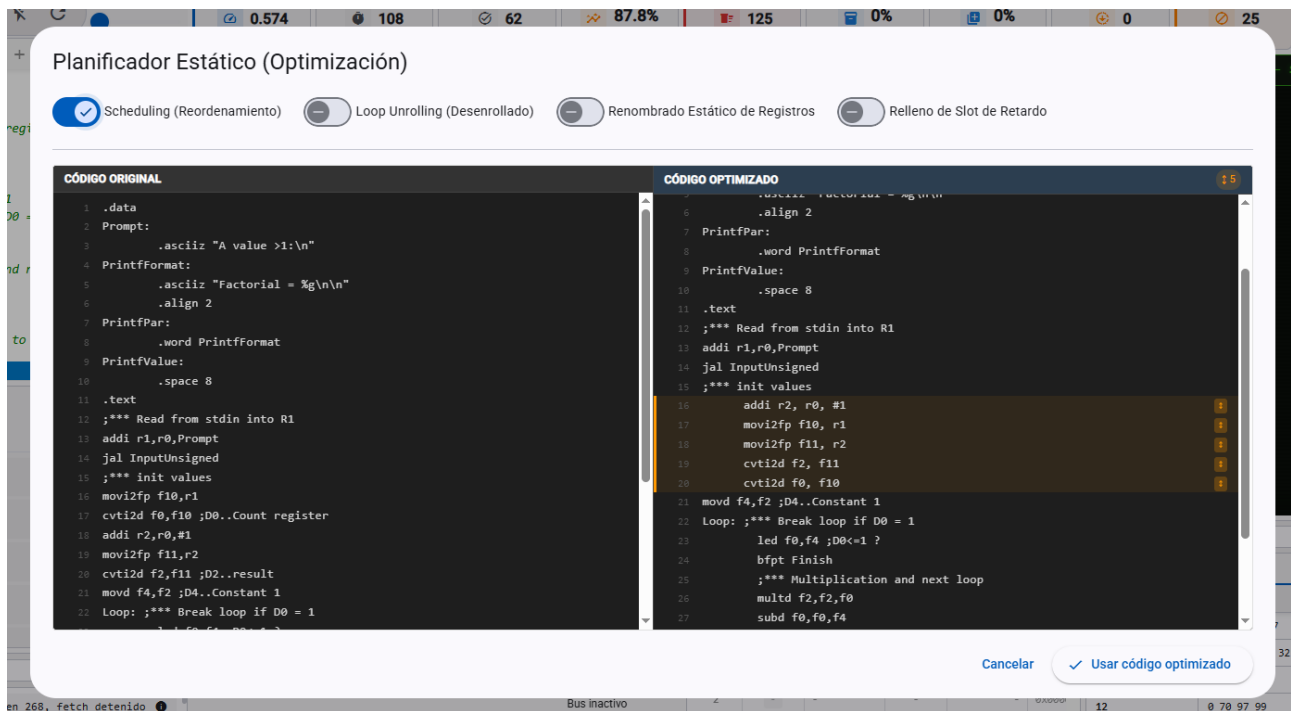


Figura 2.30: Diálogo con el código original a la izquierda y el optimizado con las técnicas seleccionadas a la derecha.

El diálogo muestra el código original a la izquierda y el optimizado a la derecha. Al pulsar el botón “Usar código optimizado”, el código optimizado sustituye al de la pestaña activa del editor. Estas optimizaciones son estáticas (se aplican antes de ejecutar) y son independientes de la configuración de la máquina (salvo el relleno del slot de retardo, que requiere tener activado el Branch Delay Slot en la configuración del procesador).

2.5. Excepciones

El simulador implementa un sistema de excepciones que gestiona los errores que pueden producirse durante las ejecuciones de los programas. Esto es válido para cualquier tipo de procesador seleccionado. El procedimiento es el siguiente:

- Ocurre una excepción durante la ejecución de una instrucción.
- Cuando la instrucción llega a la etapa de escritura (WB, o RI en el superescalar), el simulador guarda en el registro “IAR” la dirección de la instrucción que ha provocado la excepción y en el registro “CAUSE” el código de la excepción.
- Se descartan las instrucciones posteriores que estuvieran en el pipeline.
- El simulador carga en el PC la dirección de la entrada de la tabla de vectores correspondiente a ese código de excepción.
- Esa entrada contiene una instrucción de salto (J) al manejador, que se ejecuta a continuación.

- El manejador puede terminar con la instrucción RFE, que continúa la ejecución en la instrucción siguiente a la que provocó la excepción.

Los tipos de excepciones soportados son:

- Desbordamiento aritmético.
- División entera por cero.
- División por cero en punto flotante.
- Desbordamiento (overflow) en punto flotante.
- Subdesbordamiento (underflow) en punto flotante.
- Operación inválida en punto flotante (por ejemplo, un resultado NaN).
- Dirección de memoria no alineada.
- Dirección de memoria fuera de límites.
- Instrucción ilegal.

Las llamadas al sistema (TRAP) no son excepciones: el simulador las ejecuta directamente, sin pasar por la tabla de vectores.

Las excepciones son precisas: el estado del procesador refleja exactamente el punto donde ocurrió la excepción.

2.5.1. Tabla de Vectores

La tabla de vectores de excepción se genera en las primeras direcciones de la memoria de instrucciones (0x00 a 0x5F). Cada entrada ocupa 4 bytes y contiene una instrucción de salto (J) al manejador correspondiente:

- **0x00:** Reset.
- **0x04:** Overflow aritmético.
- **0x08:** División por cero.
- **0x0C:** División por cero flotante.
- **0x10:** Overflow flotante.
- **0x14:** Underflow flotante.
- **0x18:** Operación inválida flotante.
- **0x1C:** Dirección desalineada.

- **0x20**: Dirección fuera de límites.
- **0x24**: Instrucción ilegal.
- **0x28**: Trap 0.
- **0x2C**: Trap 1.
- **0x30**: Trap 2.
- **0x34**: Trap 3.
- **0x38**: Trap 4.
- **0x3C**: Trap 5.
- **0x40-0x5C**: Reservados a futuro.

A continuación de la tabla, en la dirección **0x60**, se sitúa el manejador por defecto (una instrucción TRAP 6), y el código del usuario comienza en la dirección **0x64**.

Cuando ocurre una excepción, el procesador salta a la entrada de la tabla correspondiente y la instrucción J que contiene lo lleva al manejador. Las entradas de los TRAP (0x28 a 0x3C) están reservadas: actualmente el procesador ejecuta los TRAP directamente y no salta a ellas.

2.5.2. Manejadores predefinidos

El simulador reconoce etiquetas especiales para los manejadores de excepción, que si se incluyen en el programa del usuario, permiten sobrescribir el manejador por defecto. Estas etiquetas de manejadores son:

- `__reset_handler.`
- `__overflow_handler.`
- `__divzero_handler.`
- `__fp_divzero_handler.`
- `__fp_overflow_handler.`
- `__fp_underflow_handler.`
- `__fp_invalid_handler.`
- `__mem_align_handler.`
- `__mem_bounds_handler.`
- `__illegal_instr_handler.`
- `__trap_exit_handler.`

- `__trap_open_handler`.
- `__trap_close_handler`.
- `__trap_read_handler`.
- `__trap_write_handler`.
- `__trap_printf_handler`.

Las excepciones que no tienen un manejador definido saltan al manejador por defecto (`__default_handler`, en la dirección 0x60), que genera el propio ensamblador y siempre detiene el programa. Para tratar una excepción hay que definir su manejador específico.

Cuando se define una de estas etiquetas en el código de usuario, la instrucción de salto de la entrada correspondiente de la tabla de vectores apunta al manejador del usuario, por lo que si ocurre una excepción de este tipo, el programa saltará a él, permitiéndole el control de la excepción. Las etiquetas de los TRAP (`__trap*_handler`) se reconocen, pero actualmente no tienen efecto, ya que los TRAP no pasan por la tabla de vectores.

2.5.3. Manejador por defecto

Por defecto, cuando ocurre una excepción y no hay un manejador declarado en el código del usuario, el simulador ejecuta el manejador por defecto `__default_handler`, situado en la dirección 0x60. Este manejador ejecuta TRAP 6, que detiene inmediatamente la ejecución del programa. El tipo de excepción ocurrida se puede consultar en el panel de “Log de eventos”, que muestra el mensaje “EXCEPCIÓN PRECISA” junto a la dirección de la instrucción y el código de la excepción. La consola de entrada / salida no muestra ningún mensaje.

2.5.4. Registro IAR

Registro especial de 32 bits de la arquitectura que el propio procesador escribe automáticamente: al producirse una excepción guarda la dirección de la instrucción que la ha provocado, y al ejecutar un TRAP guarda la dirección de la instrucción siguiente al TRAP. Solo puede escribirlo el propio procesador: ninguna instrucción de usuario puede modificarlo. Se muestra en el panel de registros del simulador, al final del banco de enteros.

2.5.5. Registro CAUSE

Registro especial interno que almacena el código que identifica el tipo de excepción ocurrida. Lo actualiza el procesador al producirse una excepción y se borra al ejecutar RFE. No se muestra en el panel de registros ni puede leerse desde ninguna instrucción; el código de la excepción se puede consultar en el “Log de eventos”. Los valores posibles son:

- `ARITHMETIC_OVERFLOW`.

- DIVISION_BY_ZERO.
- FP_DIVISION_BY_ZERO.
- FP_OVERFLOW.
- FP_UNDERFLOW.
- FP_INVALID_OPERATION.
- MEMORY_ALIGNMENT_ERROR.
- MEMORY_OUT_OF_BOUNDS.
- ILLEGAL_INSTRUCTION.
- UNKNOWN.

2.5.6. Instrucción RFE

Esta instrucción especial permite retornar de un manejador de excepciones. Carga en el PC la dirección guardada en el IAR más 4 ($PC \leftarrow IAR + 4$) y borra el registro CAUSE. Como el IAR contiene la dirección de la instrucción que provocó la excepción, la ejecución continúa en la instrucción *siguiente* a ella: la instrucción que falló no se vuelve a ejecutar.

2.6. Planificación estática

2.6.1. Reordenamiento de instrucciones (Scheduling)

Esta técnica de reordenamiento busca minimizar los ciclos de parada (stall). Divide el código en bloques básicos (hasta una instrucción de salto) y analiza, por cada bloque básico, las dependencias de datos (RAW) y las latencias de las unidades para alterar el orden en el programa, haciendo que el procesador las ejecute más rápido sin cambiar el resultado final. El Listado 2.1 presenta un código con posibles optimizaciones y el mismo código optimizado tras aplicar reordenamiento.

1	LW	R1, 0(R2)	; 1. Carga el dato de memoria en R1 (
			Supongamos que tarda 2 ciclos)
2	ADD	R3, R1, R4	; 2. STALL: R1 no está listo. El procesador
			se detiene 1 ciclo.
3	LW	R5, 4(R2)	; 3. Carga otro dato en R5
4	ADD	R6, R5, R7	; 4. STALL: R5 no está listo. El procesador
			se detiene 1 ciclo.
5			
6			; Optimizado
7	LW	R1, 0(R2)	; 1. Inicia la carga de R1
8	LW	R5, 4(R2)	; 2. Intercalada: Inicia la carga de R5 (
			Mientras R1 se procesa)
9	ADD	R3, R1, R4	; 3. R1 ya está listo. No hay parada.

```
10  ADD    R6, R5, R7    ; 4. R5 ya está listo. No hay parada.
```

Listado 2.1: Ejemplo de reordenamiento de instrucciones para optimizar un código.

2.6.2. Desenrollado de bucles (Loop unrolling)

Esta técnica consiste en replicar el cuerpo de un bucle N veces. Esto reduce las instrucciones de salto, permitiendo además al planificador encontrar más instrucciones independientes para reordenar. Un factor de cuatro, por ejemplo, replica el cuerpo del bucle cuatro veces y modifica los incrementos de los punteros en cuatro. Esta técnica aumenta el paralelismo a costa de incrementar el tamaño del binario del programa. El Listado 2.2 muestra el código de un bucle optimizado.

```
1  ;codigo original
2  .text
3  ADDI R1, R0, #10
4  LOOP: SUBI R1, R1, #1
5  BNEZ R1, LOOP
6
7  ; bucle desenrollado
8  .text
9  ADDI R1, R0, #10
10 LOOP: SUBI R1, R1, #8
11 BNEZ R1, LOOP
```

Listado 2.2: Ejemplo de desenrollado de bucle con factor 8.

2.6.3. Renombrado estático de registros

Esta técnica trata de eliminar las dependencias WAR y WAW cambiando los registros usados por instrucciones dependientes por otros que estén libres, lo que permite a su vez al planificador mover instrucciones con mayor libertad. El Listado 2.3 muestra la aplicación de esta técnica a un código de ejemplo.

```
1  ; Original (Riesgo WAR)
2  ADD R1, R2, R3
3  SUB R2, R4, R5    ; R2 se usa antes de escribirse
4
5  ; Optimizado
6  ADD R1, R2, R3
7  SUB R10, R4, R5   ; Usa R10 en lugar de R2 para eliminar la
   dependencia
```

Listado 2.3: Ejemplo de renombrado de registros.

2.6.4. Relleno de slot de retardo

Esta técnica depende de que la arquitectura permita la ejecución de `Branch delay slot`. El planificador busca una instrucción útil después de un salto y la mueve a la posición inmediatamente posterior al salto, haciendo que esta instrucción se ejecute siempre independientemente de la resolución del salto. Si no hay una candidata segura, el planificador inserta una instrucción `NOP`.

2.7. Ejemplos de código

Para finalizar, se incluyen varias aplicaciones de ejemplo a modo de demostración.

2.7.1. Cálculo del factorial de un número

Este ejemplo es una adaptación del manual de WinDLX. Pide un número por pantalla y calcula su factorial. Utiliza una librería adicional para obtención de números por consola. El Listado 2.4 presenta el código del programa y el Listado 2.5 presenta el código de la librería `input`.

```
1 .data
2   Prompt:
3   .asciiz "A value >1:\n"
4   PrintfFormat:
5   .asciiz "Factorial = %g\n\n"
6   .align 2
7   PrintfPar:
8   .word PrintfFormat
9   PrintfValue:
10  .space 8
11 .text
12  ;*** Read from stdin into R1
13  addi r1,r0,Prompt
14  jal InputUnsigned
15  ;*** init values
16  movi2fp f10,r1
17  cvti2d f0,f10 ;D0..Count register
18  addi r2,r0,#1
19  movi2fp f11,r2
20  cvti2d f2,f11 ;D2..result
21  movd f4,f2 ;D4..Constant 1
22  Loop: ;*** Break loop if D0 = 1
23      led f0,f4 ;D0<=1 ?
24      bfpt Finish
25      ;*** Multiplication and next loop
26      mulld f2,f2,f0
27      subd f0,f0,f4
28      j Loop
29  Finish: ;*** write result to stdout
30      sd PrintfValue,f2
31      addi r14,r0,PrintfPar
32      trap 5
```

33 trap 0

Listado 2.4: Programa factorial.

```
1 .data
2   ;*** Data for Read-Trap
3   ReadBuffer: .space 80
4   ReadPar: .word 0,ReadBuffer,80
5   ;*** Data for Printf-Trap
6   PrintfPar: .space 4
7   Saver2: .space 4
8   Saver3: .space 4
9   Saver4: .space 4
10  Saver5: .space 4
11 .text
12 .global InputUnsigned
13 InputUnsigned:
14   ;*** save register contents
15   sw Saver2,r2
16   sw Saver3,r3
17   sw Saver4,r4
18   sw Saver5,r5
19   ;*** Prompt
20   sw PrintfPar,r1
21   addi r14,r0,PrintfPar
22   trap 5
23   ;*** call Trap-3 to read line
24   addi r14,r0,ReadPar
25   trap 3
26   ;*** determine value
27   addi r2,r0,ReadBuffer
28   addi r1,r0,#0
29   addi r4,r0,#10 ;Decimal system
30   Loop: ;*** reads digits to end of line
31       lbu r3,0(r2)
32       seqi r5,r3,#10 ;LF -> Exit
33       bnez r5,Finish
34       subi r3,r3,#48 ; 0
35       multu r1,r1,r4 ;Shift decimal
36       add r1,r1,r3
37       addi r2,r2,#1 ;increment pointer
38       j Loop
39   Finish: ;*** restore old register contents
40       lw r2,Saver2
41       lw r3,Saver3
42       lw r4,Saver4
43       lw r5,Saver5
44       jr r31 ; Return
```

Listado 2.5: Librería input.

2.7.2. Test de número primo

Este ejemplo pide por consola un número y calcula si se trata de un número primo o no. El Listado 2.6 presenta el código del programa.

```
1  .data
2  msg_prompt:  .ascii "Introduce un numero (2-99): "
3  .align 2
4  msg_prime:   .ascii "El numero %d es PRIMO.\n"
5  .align 2
6  msg_not:     .ascii "El numero %d NO es primo.\n"
7  .align 2
8  buffer:      .space 20
9
10 .align 2
11 p_read:       .word 0, buffer, 10
12 .align 2
13 p_print:      .word 0, 0
14
15 .text
16 main:
17 ; 1. Mostrar prompt inicial
18 addi r1, r0, msg_prompt
19 sw    p_print, r1
20 addi r14, r0, p_print
21 trap 5
22
23 ; 2. Leer numero del usuario
24 addi r14, r0, p_read
25 trap 3
26
27 ; 3. Convertir ASCII a Entero (atoi)
28 addi r2, r0, buffer      ; R2 = puntero al buffer
29 addi r3, r0, #0          ; R3 = acumulador
30
31 loop_atoi:
32 lb    r4, 0(r2)           ; Leer byte
33 addi r5, r4, #-48         ; ASCII a int ('0'=48)
34 bltz r5, end_atoi
35 addi r6, r0, #9
36 sub   r6, r6, r5
37 bltz r6, end_atoi
38
39 ; acumulador = (acumulador * 10) + digito
40 slli r7, r3, #3           ; *8
41 slli r8, r3, #1           ; *2
42 add   r3, r7, r8          ; *10
43 add   r3, r3, r5
44
45 addi r2, r2, #1
46 j     loop_atoi
47
48 end_atoi:
49 ; Probar si R3 es primo
50 addi r6, r0, #2
51 sub   r7, r3, r6
52 bltz r7, is_not_prime
53
```

```
54      addi r10, r0, #2          ; divisor
55
56      check_loop:
57      sub  r7, r10, r3
58      beqz r7, is_prime
59
60      div  r11, r3, r10
61      mult r12, r11, r10
62      sub  r13, r3, r12
63
64      beqz r13, is_not_prime
65
66      addi r10, r10, #1
67      j    check_loop
68
69      is_prime:
70      addi r1, r0, msg_prime
71      j    print_result
72
73      is_not_prime:
74      addi r1, r0, msg_not
75
76      print_result:
77      sw   p_print, r1          ; fmtPtr
78      addi r10, r0, p_print
79      sw   4(r10), r3           ; arg1
80      addi r14, r0, p_print
81      trap 5
82
83      trap 0
```

Listado 2.6: Test de número primo.

2.7.3. Algoritmo de la burbuja

Clásico algoritmo que ordena los elementos de un array. El Listado 2.7 presenta el código del programa.

```
1      .data
2      .align 2
3      size:          .word 8
4      array:         .word 50, 10, 80, 20, 40, 70, 30, 60
5
6      msg_done:      .ascii "Array ordenado: %d, %d, %d, %d, %d, %d, %d, %d\
7                      n"
8      .align 2
9      p_print:       .word msg_done
10     .space 32       ; hueco para los 8 resultados
11
12     .text
13     main:
14     lw   r1, size          ; N = 8
15     addi r2, r0, #0        ; i = 0 (bucle externo)
```

```

16  outer_loop:
17  addi r3, r1, #-1          ; r3 = N - 1
18  sub  r4, r2, r3          ; i == N-1
19  beqz r4, print_array
20
21  addi r5, r0, #0          ; j = 0 (bucle interno)
22  addi r6, r1, #-1
23  sub  r6, r6, r2          ; Limite j = N - 1 - i
24
25  inner_loop:
26  sub  r7, r5, r6          ; j == Limite
27  beqz r7, next_outer
28
29  ; Cargar array[j] y array[j+1]
30  slli r8, r5, #2          ; r8 = j * 4
31  addi r9, r0, array       ; r9 = &array
32  add  r9, r9, r8          ; r9 = &array[j]
33  lw   r10, 0(r9)          ; r10 = array[j]
34  lw   r11, 4(r9)          ; r11 = array[j+1]
35
36  ; array[j] > array[j+1]
37  sub  r12, r11, r10       ; r12 = a[j+1] - a[j]
38  bgtz r12, next_inner    ; Si ok (a[j+1] > a[j]), no hacer swap
39
40  ; Intercambiar (SWAP)
41  sw   0(r9), r11          ; array[j] = r11
42  sw   4(r9), r10          ; array[j+1] = r10
43
44  next_inner:
45  addi r5, r5, #1          ; j++
46  j    inner_loop
47
48  next_outer:
49  addi r2, r2, #1          ; i++
50  j    outer_loop
51
52  print_array:
53  ; Copiar array a p_print para mostrar con TRAP 5
54  addi r1, r0, #0          ; index i = 0
55  addi r2, r0, array       ; r2 = base array
56  addi r3, r0, p_print     ; r3 = base p_print
57  copy_loop:
58  slli r4, r1, #2          ; r4 = i * 4
59  add  r5, r2, r4          ; r5 = &array[i]
60  lw   r6, 0(r5)          ; r6 = array[i]
61  add  r7, r3, r4          ; r7 = &p_print[i]
62  sw   4(r7), r6          ; Almacenar en p_print + 4 + i*4
63  addi r1, r1, #1
64  addi r8, r1, #-8
65  bltz r8, copy_loop
66
67  addi r14, r0, p_print    ; R14 apunta al bloque de parametros
68  trap 5                  ; Imprimir
69  trap 0                  ; Fin

```

Listado 2.7: Algoritmo de la burbuja que ordena un array y muestra el resultado por pantalla.

2.7.4. Serie de Fibonacci

Algoritmo que genera los 10 primeros términos de la serie. El Listado 2.8 presenta el código del programa.

```
1  .data
2  .align 2
3  n_terms:      .word 10
4  msg_item:     .ascii "Fibonacci(%d) = %d\n"
5
6  .align 2
7  p_print:      .word msg_item
8  .word 0       ; para el indice %d
9  .word 0       ; para el valor %d
10
11 .text
12 main:
13 addi r20, r0, #0      ; F(n-2) = 0
14 addi r21, r0, #1      ; F(n-1) = 1
15 addi r22, r0, #0      ; i = 0
16 lw   r23, n_terms    ; N = 10
17
18 loop:
19 sub  r5, r22, r23      ; i == N
20 beqz r5, end
21
22 ; Preparar bloque de parametros para TRAP 5
23 addi r10, r0, p_print
24 sw   4(r10), r22       ; Guardar indice i
25 sw   8(r10), r20       ; Guardar valor F(n-2)
26
27 addi r14, r0, p_print
28 trap 5                ; Imprimir "Fibonacci(i) = F(n-2)"
29
30 ; Calcular siguiente: r6 = r20 + r21
31 add  r6, r20, r21
32 addi r20, r21, #0      ; F(n-2) = F(n-1)
33 addi r21, r6, #0       ; F(n-1) = F(n)
34
35 addi r22, r22, #1      ; i++
36 j    loop
37
38 end:
39 trap 0
```

Listado 2.8: Algoritmo de Fibonacci.

2.7.5. Cálculo del área de un círculo

Algoritmo que pide por pantalla un número (el radio del círculo) y calcula su área. El Listado 2.9 muestra el código del programa.

```
1  .data
2  .align 3
```

```
3 pi: .double 3.14159265
4 zero: .double 0.0
5
6 msg_prompt: .ascii "Introduce radio (entero positivo): "
7 .align 2
8 msg_err: .ascii "Error: El radio debe ser positivo.\n"
9 .align 2
10 msg_res: .ascii "Area del circulo = %f\n"
11
12 .align 2
13 p_read: .word 0, buffer, 10
14
15 ; El bloque de parametros para TRAP 5 debe estar alineado
16 ; para que el double en offset 4 sea accesible correctamente.
17 .align 3
18 p_print: .word 0 ; [Offset 0] fmtPtr
19 .word 0 ; [Offset 4] arg1 (High 32 bits de double)
20 .word 0 ; [Offset 8] arg1 (Low 32 bits de double)
21
22 buffer: .space 20
23
24 .text
25 main:
26 ; 1. Cargar PI y Zero
27 ld f10, pi
28 ld f12, zero
29
30 ; 2. Mostrar prompt
31 addi r1, r0, msg_prompt
32 sw p_print, r1
33 addi r14, r0, p_print
34 trap 5
35
36 ; 3. Leer radio del usuario
37 addi r14, r0, p_read
38 trap 3
39
40 ; --- Convertir ASCII a Entero (atoi) ---
41 addi r2, r0, buffer
42 addi r1, r0, #0
43
44 loop_atoi:
45 lb r4, 0(r2)
46 addi r5, r4, #-48
47 bltz r5, end_atoi
48 addi r6, r0, #9
49 sub r6, r6, r5
50 bltz r6, end_atoi
51 slli r7, r1, #3
52 slli r8, r1, #1
53 add r1, r7, r8
54 add r1, r1, r5
55 addi r2, r2, #1
56 j loop_atoi
57
58 end_atoi:
59 ; 4. Convertir entero r1 a double en f0
60 movi2fp f0, r1
```

```
61    cvti2d    f0, f0
62
63    ; 5. Validar radio > 0
64    led    f0, f12
65    bfpt    error
66
67    ; 6. Calcular Area = PI * R * R
68    multd   f2, f0, f0
69    multd   f2, f2, f10
70
71    ; 7. Mostrar resultado
72    addi    r1, r0, msg_res
73    addi    r10, r0, p_print
74    sw      0(r10), r1          ; Guardar puntero al formato en offset 0
75    sd      4(r10), f2          ; Guardar double en offset 4 (ocupa 4 y 8)
76
77    addi    r14, r0, p_print
78    trap    5
79    j        end
80
81    error:
82    addi    r1, r0, msg_err
83    sw      p_print, r1
84    addi    r14, r0, p_print
85    trap    5
86
87    end:
88    trap    0
```

Listado 2.9: Cálculo del área de un círculo.

2.7.6. Operación DAXPY vectorial

Algoritmo que implementa el bucle DAXPY vectorial con vectores de 16 elementos. El Listado 2.10 muestra el código del programa.

```
1    .data
2    .align 3
3    val_a:      .double 2.5
4    vector_x:    .double 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0,
5                10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0
6    vector_y:    .double 10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0,
7                10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0
8
9    msg_res:     .ascii "Resultado Y[0] = %f (Esperado: 12.5)\n"
10   .align 2
11   p_print:      .word msg_res
12   .double 0.0
13
14   .text
15   main:
16   ; 1. Configurar VL (Vector Length) a 16
17   addi    r1, r0, #16
18   movi2s  VLR, r1          ; VLR = 16
```

```
17
18 ; 2. Cargar escalar 'a' en F0
19 ld    f0, val_a
20
21 ; 3. Cargar vectores
22 lv    v1, vector_x
23 lv    v2, vector_y
24
25 ; 4. Operacion DAXPY
26 multsv v3, f0, v1
27 addv   v2, v3, v2
28
29 ; 5. Guardar resultado final
30 sv     vector_y, v2
31
32 ; 6. Mostrar el primer elemento
33 ld     f2, vector_y
34 addi   r10, r0, p_print
35 sd     4(r10), f2
36 addi   r14, r0, p_print
37 trap  5
38
39 trap  0
```

Listado 2.10: DAXPY vectorial.

Capítulo 3

Catálogo de instrucciones

Este capítulo contiene la ISA del simulador desarrollado. Se han agrupado las instrucciones implementadas según su tipo y sus operandos.

3.1. Aritmética y lógica entera

En esta sección se listan las operaciones aritméticas y lógicas que operan con registros.

ADD — ADD Rd, Rs1, Rs2

Suma aritmética de enteros con signo. Realiza la operación $Rd = Rs1 + Rs2$. El hardware detecta desbordamiento aritmético si el resultado no puede representarse en 32 bits con signo.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.1 presenta un ejemplo de construcción de la instrucción ADD.

```
1  ADD R3, R1, R2 ; R3 = R1 + R2
```

Listado 3.1: ADD

ADDU — ADDU Rd, Rs1, Rs2

Suma aritmética de enteros sin signo. $Rd = Rs1 + Rs2$. A diferencia de ADD, esta instrucción no genera excepciones por desbordamiento (overflow); el resultado simplemente se trunca a 32 bits.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.2 presenta un ejemplo de construcción de la instrucción ADDU.

```
1  ADDU R3, R1, R2 ; Suma sin riesgo de excepcion
```

Listado 3.2: ADDU

SUB — SUB Rd, Rs1, Rs2

Resta aritmética de enteros con signo. $Rd = Rs1 - Rs2$. Genera una excepción de `ARITHMETIC_OVERFLOW` si el resultado excede el rango de 32 bits.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.3 presenta un ejemplo de construcción de la instrucción **SUB**.

```
1 SUB R3, R5, R2 ; R3 = R5 - R2
```

Listado 3.3: SUB

SUBU — SUBU Rd, Rs1, Rs2

Resta aritmética de enteros sin signo. $Rd = Rs1 - Rs2$. A diferencia de **SUB**, esta instrucción no genera excepciones por desbordamiento (overflow); el resultado simplemente se trunca a 32 bits.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.4 presenta un ejemplo de construcción de la instrucción **SUBU**.

```
1 SUBU R3, R5, R2 ; R3 = R5 - R2
```

Listado 3.4: SUBU

MULT — MULT Rd, Rs1, Rs2

Multiplicación de enteros de 32 bits con signo. Esta operación por defecto tiene una latencia mayor, de 7 ciclos. Esta instrucción no genera excepciones por desbordamiento (overflow); el resultado simplemente se trunca a 32 bits.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.5 presenta un ejemplo de construcción de la instrucción **MULT**.

```
1 MULT R4, R2, R3 ; R4 = R2 * R3
```

Listado 3.5: MULT

MULTU — MULTU Rd, Rs1, Rs2

Multiplicación de enteros sin signo. Esta operación por defecto tiene una latencia mayor, de 7 ciclos. Esta instrucción no genera excepciones por desbordamiento (overflow); el resultado simplemente se trunca a 32 bits.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.6 presenta un ejemplo de construcción de la instrucción **MULTU**.

```
1  MULTU R4, R2, R3 ; R4 = R2 * R3
```

Listado 3.6: MULTU

DIV — DIV Rd, Rs1, Rs2

División entera con signo. $Rd = Rs1 / Rs2$. Es la operación entera más lenta, por defecto tiene una latencia de 24 ciclos.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.7 presenta un ejemplo de construcción de la instrucción **DIV**.

```
1  DIV R4, R6, R2 ; R4 = R6 / R2
```

Listado 3.7: DIV

Notas: Si el divisor (Rs2) es cero, se genera una excepción de **DIVISION_BY_ZERO**.

DIVU — DIVU Rd, Rs1, Rs2

División entera sin signo. $Rd = Rs1 / Rs2$ tratando ambos operandos como valores sin signo de 32 bits. Es la operación entera más lenta, por defecto tiene una latencia de 24 ciclos.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.8 presenta un ejemplo de construcción de la instrucción **DIVU**.

```
1  DIVU R4, R6, R2 ; R4 = R6 / R2
```

Listado 3.8: DIVU

Notas: Si el divisor (Rs2) es cero, se lanza una excepción de **DIVISION_BY_ZERO**.

AND — AND Rd, Rs1, Rs2

Operación lógica AND bit a bit.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.9 presenta un ejemplo de construcción de la instrucción **AND**.

```
1  AND R4, R1, R2 ; R4 = R1 & R2
```

Listado 3.9: AND

OR — OR Rd, Rs1, Rs2

Operación lógica OR bit a bit.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.10 presenta un ejemplo de construcción de la instrucción OR.

```
1 OR R4, R1, R2 ; R4 = R1 | R2
```

Listado 3.10: OR

XOR — XOR Rd, Rs1, Rs2

Operación lógica XOR (OR exclusiva) bit a bit.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Segundo operando fuente. El Listado 3.11 presenta un ejemplo de construcción de la instrucción XOR.

```
1 XOR R4, R1, R2 ; R4 = R1 ^ R2
```

Listado 3.11: XOR

SLL — SLL Rd, Rs1, Rs2

Desplazamiento lógico a la izquierda (Shift Left Logical). El número de posiciones se toma de los 5 bits bajos de Rs2.

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Registro cuyos 5 bits bajos indican el número de posiciones a desplazar. El Listado 3.12 presenta un ejemplo de construcción de la instrucción SLL.

```
1 SLL R4, R1, R2 ; R4 = R1 << R2
```

Listado 3.12: SLL

SRL — SRL Rd, Rs1, Rs2

Desplazamiento lógico a la derecha (rellena con ceros).

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Registro cuyos 5 bits bajos indican el número de posiciones a desplazar. El Listado 3.13 presenta un ejemplo de construcción de la instrucción SRL.

```
1 SRL R4, R1, R2 ; R4 = R1 >> R2
```

Listado 3.13: SRL

SRA — SRA Rd, Rs1, Rs2

Desplazamiento aritmético a la derecha (mantiene el bit de signo).

Operandos: Rd: Registro destino. Rs1: Primer operando fuente. Rs2: Registro cuyos 5 bits bajos indican el número de posiciones a desplazar. El Listado 3.14 presenta un ejemplo de construcción de la instrucción SRA.

```
1 SRA R4, R1, R2
```

Listado 3.14: SRA

NOP — NOP

No Operación. Instrucción nula que no altera el estado de los registros ni la memoria. El Listado 3.15 presenta un ejemplo de construcción de la instrucción NOP.

```
1 NOP ; Relleno para slots de retardo o alineacion
```

Listado 3.15: NOP

Notas: Es fundamental en la planificación estática para rellenar huecos que el compilador no ha podido optimizar.

3.2. Aritmética y lógica entera con inmediatos

En esta sección se listan las operaciones aritméticas y lógicas en las que uno de sus operandos es un entero inmediato.

ADDI — ADDI Rd, Rs1, #imm

Suma Rs1 con un inmediato de 16 bits con extensión de signo y guarda el resultado en Rd.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits con signo. El prefijo # es obligatorio para literales numéricos. Acepta decimal (#10, #-5) o hexadecimal (#0xFF). También se puede especificar una etiqueta (sin #), que el ensamblador resolverá a su dirección (debe estar definida en `.data`, no en `.text`). El Listado 3.16 presenta un ejemplo de construcción de la instrucción ADDI.

```
1 ADDI R3, R1, #10           ; R3 = R1 + 10 (decimal)
2 ADDI R3, R1, #0xFF        ; R3 = R1 + 255 (hexadecimal)
3 ADDI R3, R1, MiVar         ; R3 = R1 + direccion de MiVar
```

Listado 3.16: ADDI

Notas: Comúnmente usada para cargar constantes pequeñas o calcular desplazamientos. El inmediato se extiende a 32 bits con signo antes de sumar. Igual que ADD, genera una excepción de ARITHMETIC_OVERFLOW si el resultado excede el rango de 32 bits con signo.

ADDUI — ADDUI Rd, Rs1, #imm

Suma Rs1 con un inmediato de 16 bits sin signo (extensión de ceros) y guarda el resultado en Rd.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits sin signo, decimal (#10) o hexadecimal (#0x100), o etiqueta de .data sin #. El Listado 3.17 presenta un ejemplo de construcción de la instrucción ADDUI.

1	ADDUI R3, R1, #10	; R3 = R1 + 10
2	ADDUI R3, R1, #0x100	; R3 = R1 + 256

Listado 3.17: ADDUI

Notas: A diferencia de ADDI, el inmediato se extiende con ceros y se trata siempre como positivo (unsigned). El ensamblador acepta valores entre -32768 y 65535 , pero un valor negativo se reinterpreta como su equivalente de 16 bits sin signo: por ejemplo, #-5 se suma como 65531.

SUBI — SUBI Rd, Rs1, #imm

Resta un inmediato de 16 bits con signo a Rs1 y guarda el resultado en Rd.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits con signo, decimal (#10) o hexadecimal (#0x10), o etiqueta de .data sin #. El Listado 3.18 presenta un ejemplo de construcción de la instrucción SUBI.

1	SUBI R3, R1, #4	; R3 = R1 - 4
2	SUBI R3, R1, #0x10	; R3 = R1 - 16

Listado 3.18: SUBI

Notas: Habitualmente usada para restar constantes pequeñas o ajustar direcciones calculadas. El inmediato se extiende a 32 bits con signo antes de restar. Igual que SUB, genera una excepción de ARITHMETIC_OVERFLOW si el resultado excede el rango de 32 bits con signo.

SUBUI — SUBUI Rd, Rs1, #imm

Resta un inmediato de 16 bits sin signo a Rs1, tratando ambos como valores sin signo de 32 bits. $Rd = (Rs1 - imm) \bmod 2^{32}$.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits sin signo

(extensión de ceros), decimal (`#10`) o hexadecimal (`#0x100`), o etiqueta de `.data` sin `#`. El Listado 3.19 presenta un ejemplo de construcción de la instrucción `SUBUI`.

1	<code>SUBUI R3, R1, #4</code>	<code>; R3 = R1 - 4 (unsigned)</code>
2	<code>SUBUI R3, R1, #0x10</code>	<code>; R3 = R1 - 16 (unsigned)</code>

Listado 3.19: `SUBUI`

Notas: A diferencia de `SUBI`, el inmediato se extiende con ceros y no se detecta overflow; el resultado se trunca a 32 bits sin signo.

MULTI — `MULTI Rd, Rs1, #imm`

Multiplicación entera con inmediato de 16 bits con signo. $Rd = Rs1 * imm$.

Operandos: `Rd`: Registro destino. `Rs1`: Registro fuente. `#imm`: Inmediato de 16 bits con signo, decimal (`#10`) o hexadecimal (`#0x10`), o etiqueta de `.data` sin `#`. El Listado 3.20 presenta un ejemplo de construcción de la instrucción `MULTI`.

1	<code>MULTI R3, R1, #10</code>	<code>; R3 = R1 * 10</code>
---	--------------------------------	-----------------------------

Listado 3.20: `MULTI`

Notas: Latencia de 7 ciclos. El inmediato se extiende a 32 bits con signo.

DIVI — `DIVI Rd, Rs1, #imm`

División entera con inmediato de 16 bits con signo. $Rd = Rs1 / imm$.

Operandos: `Rd`: Registro destino. `Rs1`: Registro fuente. `#imm`: Inmediato de 16 bits con signo, decimal (`#10`) o hexadecimal (`#0x10`), o etiqueta de `.data` sin `#`. El Listado 3.21 presenta un ejemplo de construcción de la instrucción `DIVI`.

1	<code>DIVI R3, R6, #2</code>	<code>; R3 = R6 / 2</code>
---	------------------------------	----------------------------

Listado 3.21: `DIVI`

Notas: Latencia de 24 ciclos. Si el inmediato es 0, se lanza una excepción de `DIVISION_BY_ZERO`.

ANDI — `ANDI Rd, Rs1, #imm`

AND lógico bit a bit entre `Rs1` y un inmediato de 16 bits (extensión de ceros).

Operandos: `Rd`: Registro destino. `Rs1`: Registro fuente. `#imm`: Inmediato de 16 bits sin signo (extensión de ceros), decimal (`#10`) o hexadecimal (`#0x100`), o etiqueta de `.data` sin `#`. El Listado 3.22 presenta un ejemplo de construcción de la instrucción `ANDI`.

1	ANDI R4, R1, #0xFF	; R4 = R1 & 0xFF (mascara 8 bits bajos)
2	ANDI R4, R1, #0xF0	; R4 = R1 & 0xF0 (mascara nibble alto)

Listado 3.22: ANDI

Notas: Muy útil para enmascarar bits. El inmediato se extiende a 32 bits con ceros.

ORI — ORI Rd, Rs1, #imm

OR lógico bit a bit entre Rs1 y un inmediato de 16 bits (extensión de ceros).

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits sin signo (extensión de ceros), decimal (#10) o hexadecimal (#0x100), o etiqueta de `.data` sin `#`. El Listado 3.23 presenta un ejemplo de construcción de la instrucción ORI.

1	ORI R4, R1, #0x80	; activa el bit 7
2	ORI R1, R1, #0x5678	; combina con LHI para construir constantes de 32 bits

Listado 3.23: ORI

Notas: Frecuentemente combinada con LHI para cargar constantes de 32 bits: LHI carga los 16 bits altos y ORI rellena los 16 bajos.

XORI — XORI Rd, Rs1, #imm

XOR lógico bit a bit entre Rs1 y un inmediato de 16 bits (extensión de ceros).

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits sin signo (extensión de ceros), decimal (#10) o hexadecimal (#0x100), o etiqueta de `.data` sin `#`. El Listado 3.24 presenta un ejemplo de construcción de la instrucción XORI.

1	XORI R4, R1, #0x01	; invierte el bit 0 de R1
2	XORI R4, R1, #0xFF	; invierte los 8 bits bajos

Listado 3.24: XORI

Notas: Útil para invertir bits concretos sin afectar al resto (máscara XOR).

SLLI — SLLI Rd, Rs1, #imm

Desplazamiento lógico a la izquierda de Rs1 por #imm posiciones. El prefijo `#` es obligatorio para literales numéricos, en decimal o hexadecimal. Como el resto de instrucciones con inmediato, también acepta una etiqueta de `.data`, aunque en un desplazamiento rara vez tiene sentido.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Número de posiciones a desplazar, en decimal o hexadecimal. El Listado 3.25 presenta un ejemplo de construcción de la instrucción SLLI.

1	SLLI R4, R1, #2	; R4 = R1 << 2 (multiplicar por 4)
2	SLLI R4, R1, #0x3	; R4 = R1 << 3 (multiplicar por 8)

Listado 3.25: SLLI

Notas: Los bits que salen del registro se pierden. Rellena con ceros por la derecha.

SRLI — SRLI Rd, Rs1, #imm

Desplazamiento lógico a la derecha de Rs1 por #imm posiciones (rellena con ceros por la izquierda).

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Número de posiciones a desplazar, en decimal o hexadecimal. El Listado 3.26 presenta un ejemplo de construcción de la instrucción SRLI.

1	SRLI R4, R1, #3	; R4 = R1 >> 3 (division entre 8 sin signo)
---	-----------------	---

Listado 3.26: SRLI

Notas: Rellena con ceros independientemente del signo del operando. Para división con signo usar SRAI.

SRAI — SRAI Rd, Rs1, #imm

Desplazamiento aritmético a la derecha de Rs1 por #imm posiciones (propaga el bit de signo).

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Número de posiciones a desplazar, en decimal o hexadecimal. El Listado 3.27 presenta un ejemplo de construcción de la instrucción SRAI.

1	SRAI R4, R1, #1	; R4 = R1 >> 1 (division entre 2 con signo)
---	-----------------	---

Listado 3.27: SRAI

Notas: Mantiene el bit más significativo (signo) al desplazar a la derecha. Equivalente a división entera por 2^{imm} preservando el signo.

LHI — LHI Rd, #imm

Carga el inmediato en los 16 bits altos de Rd (bits 31-16). Los 16 bits bajos se ponen a 0. Útil para construir constantes de 32 bits junto con ORI.

Operandos: Rd: Registro destino. #imm: Inmediato de 16 bits sin signo que se carga en los 16 bits altos de Rd, decimal o hexadecimal (#0x1234). El Listado 3.28 presenta un ejemplo de construcción de la instrucción LHI.

1	LHI R1, #0x1234	; R1 = 0x12340000
2	ORI R1, R1, #0x5678	; R1 = 0x12345678

Listado 3.28: LHI

Notas: Patrón estándar para cargar una constante de 32 bits: usar LHI con los 16 bits altos seguido de ORI con los 16 bajos.

3.3. Comparaciones

En esta sección se listan todas las operaciones de comparación con enteros, tanto con registros como con inmediatos.

SEQ / SEQU — SEQ Rd, Rs1, Rs2 / SEQU Rd, Rs1, Rs2

Rd = 1 si Rs1 == Rs2, en caso contrario Rd = 0. SEQ es con signo, SEQU es sin signo.

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.29 presenta un ejemplo de construcción de la instrucción SEQ.

1	SEQ R5, R1, R2	; R5 = (R1 == R2) ? 1 : 0
---	----------------	---------------------------

Listado 3.29: SEQ

SNE / SNEU — SNE Rd, Rs1, Rs2

Rd = 1 si Rs1 != Rs2.

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.30 presenta un ejemplo de construcción de la instrucción SNE.

1	SNE R5, R1, R2
---	----------------

Listado 3.30: SNE

SLT / SLTU — SLT Rd, Rs1, Rs2

Rd = 1 si Rs1 < Rs2 (con signo para SLT, sin signo para SLTU).

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.31 presenta un ejemplo de construcción de la instrucción SLT.

1	SLT R5, R1, R2 ; R5 = (R1 < R2) ? 1 : 0
---	---

Listado 3.31: SLT

SGT / SGTU — SGT Rd, Rs1, Rs2

Rd = 1 si $Rs1 > Rs2$.

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.32 presenta un ejemplo de construcción de la instrucción SGT.

1	SGT R5, R1, R2
---	----------------

Listado 3.32: SGT

SLE / SLEU — SLE Rd, Rs1, Rs2

Rd = 1 si $Rs1 \leq Rs2$.

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.33 presenta un ejemplo de construcción de la instrucción SLE.

1	SLE R5, R1, R2
---	----------------

Listado 3.33: SLE

SGE / SGEU — SGE Rd, Rs1, Rs2

Rd = 1 si $Rs1 \geq Rs2$.

Operandos: Rd: Registro destino. Rs1: Primer operando. Rs2: Segundo operando. El Listado 3.34 presenta un ejemplo de construcción de la instrucción SGE.

1	SGE R5, R1, R2
---	----------------

Listado 3.34: SGE

SEQUI / SEQUI — SEQUI Rd, Rs1, #imm

Comparación de igualdad con inmediato de 16 bits. SEQUI usa extensión de signo; SEQUI usa extensión de ceros.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits (con signo en SEQI, sin signo en SEQUI). El Listado 3.35 presenta un ejemplo de construcción de la instrucción SEQI.

```
1  SEQI R5, R1, #10 ; R5 = (R1 == 10) ? 1 : 0
```

Listado 3.35: SEQI

SLTI / SLTUI / SGTI / SGTUI / SLEI / SLEUI / SGEI / SGEUI / SNEI / SNEUI — **SLTI Rd, Rs1, #imm**

Comparaciones con inmediato en todas sus variantes (menor, mayor, menor o igual, mayor o igual, distinto). Los nombres terminados en UI usan operandos sin signo.

Operandos: Rd: Registro destino. Rs1: Registro fuente. #imm: Inmediato de 16 bits. En SLTI, SGTI, SLEI, SGEI y SNEI se extiende con signo y la comparación es con signo; en las variantes terminadas en UI se extiende con ceros y la comparación es sin signo. El Listado 3.36 presenta un ejemplo de construcción de la instrucción SLTI.

```
1  SLTI R5, R1, #100 ; R5 = (R1 < 100) ? 1 : 0
```

Listado 3.36: SLTI

3.4. Aritmética, comparaciones y conversiones de punto flotante

En esta sección se listan las operaciones de punto flotante que operan con registros de tipo float, tanto las operaciones aritméticas como las comparaciones y las conversiones entre registros.

ADDF / SUBF / MULTF / DIVF — **ADDF Fd, Fs1, Fs2**

Aritmética de precisión simple (32 bits IEEE 754). El resultado se guarda en Fd. Cualquier registro float F0-F31 es válido.

Operandos: Fd: Registro flotante destino (cualquiera F0-F31). Fs1: Primer operando fuente. Fs2: Segundo operando fuente. El Listado 3.37 presenta un ejemplo de construcción de la instrucción ADDF.

```
1  ADDF F1, F3, F5 ; F1 = F3 + F5 (simple precision)
```

Listado 3.37: ADDF

Notas: Latencias por defecto: ADDF/SUBF 2 ciclos, MULTF 7 ciclos y DIVF 24 ciclos.

ADDD / SUBD / MULTD / DIVD — ADDD Fd, Fs1, Fs2

Aritmética de precisión doble (64 bits IEEE 754). Los registros deben ser PARES (F0, F2, F4...).

Operandos: Fd: Registro flotante destino, debe ser PAR (F0, F2, F4...). Fs1: Primer operando fuente, debe ser PAR. Fs2: Segundo operando fuente, debe ser PAR. El Listado 3.38 presenta un ejemplo de construcción de la instrucción MULTD.

1	MULTD F0, F2, F4 ; F0 = F2 * F4 (doble precision)
---	---

Listado 3.38: MULTD

Notas: Un double ocupa el par de registros Fn y Fn+1. En la instrucción solo se especifica Fn (el par). Latencias por defecto: ADDD/SUBD 4 ciclos, MULTD 14 ciclos y DIVD 48 ciclos.

EQF / NEF / LTF / GTF / LEF / GEF / SLTF / SGTF / SLEF / SGEF — EQF Fs1, Fs2

Comparaciones de precisión simple (32 bits). No tiene registro destino: el resultado booleano se escribe en el bit de condición de punto flotante (`fpConditionBit`) de la máquina. Usa BFPT/BFPF para saltar según el resultado. Mnemónicos: EQF (`==`), NEF (`!=`), LTF (`<`), GTF (`>`), LEF (`<=`), GEF (`>=`); SLTF/SGTF/SLEF/SGEF son variantes “Set” equivalentes.

Operandos: Fs1: Primer operando fuente. Fs2: Segundo operando fuente. El Listado 3.39 presenta un ejemplo de construcción de la instrucción LTF.

1	LTF F1, F3 ; fpConditionBit = (F1 < F3)
2	BFPT menor ; salta si fpConditionBit = 1

Listado 3.39: LTF

Notas: No se escribe ningún registro. La instrucción BFPT salta si `fpConditionBit=1`; BFPF salta si `fpConditionBit=0`. Latencia por defecto: 2 ciclos.

EQD / NED / LTD / GTD / LED / GED / SLTD / SGTD / SLED / SGED — EQD Fs1, Fs2

Comparaciones de precisión doble (64 bits, registros pares). No tiene registro destino: el resultado se escribe en `fpConditionBit` de la máquina.

Operandos: Fs1: Primer operando fuente, debe ser PAR. Fs2: Segundo operando fuente, debe ser PAR. El Listado 3.40 presenta un ejemplo de construcción de la instrucción LED.

1	LED F0, F2 ; fpConditionBit = (F0:F1 <= F2:F3)
2	BFPT fin

Listado 3.40: LED

Notas: Los registros deben ser pares; cada uno representa un double de 64 bits junto con el siguiente registro (Fn:Fn+1). Latencia por defecto: 4 ciclos.

CVTF2D / CVTD2F — CVTF2D Fd, Fs

Conversión entre precisión simple y doble. CVTF2D convierte SP→DP; CVTD2F convierte DP→SP.

Operandos: Fd: Registro flotante destino (par en CVTF2D). Fs: Registro flotante fuente con el valor a convertir (par en CVTD2F). El Listado 3.41 presenta un ejemplo de construcción de la instrucción CVTF2D.

1	CVTF2D F0, F5 ; convierte F5 (SP) a DP en F0:F1
---	---

Listado 3.41: CVTF2D

Notas: El registro de doble precisión (destino en CVTF2D, fuente en CVTD2F) debe ser par. Latencia por defecto de todas las conversiones: 4 ciclos.

CVTI2D / CVTI2F / CVTD2I / CVTF2I — CVTI2D Fd, Fs

Conversión entre entero y flotante:

- CVTI2D: entero → double.
- CVTI2F: entero → simple.
- CVTD2I: double → entero truncado.
- CVTF2I: simple → entero truncado.

Operandos: Fd: Registro flotante destino. Siempre es un registro Fd, incluso en CVTD2I y CVTF2I (el valor entero truncado se guarda en el banco de punto flotante, no en el de enteros). Fs: Registro flotante fuente con el valor a convertir. El Listado 3.42 presenta un ejemplo de construcción de la instrucción CVTI2D.

1	CVTI2D F0, F10 ; F0:F1 = double(entero en F10)
2	CVTD2I F3, F0 ; F3 = trunc(F0:F1 como double)

Listado 3.42: CVTI2D

Notas: CVTD2I y CVTF2I escriben su resultado en un registro flotante (Fd), no en uno entero, pese a que el resultado sea un valor entero truncado: el ensamblador exige la sintaxis CVTD2I Fd, Fs. Para llevar ese valor a un registro entero (Rd) hace falta un MOVFP2I posterior. El destino DP de CVTI2D y la fuente DP de CVTD2I deben ser registros pares.

MOVF / MOVD — MOVF Fd, Fs

Copia un registro flotante. MOVF copia precisión simple; MOVD copia un par DP (Fd y Fd+1 $\leftarrow$ Fs y Fs+1).

Operandos: Fd: Registro flotante destino (par para MOVD). Fs: Registro fuente (par para MOVD). El Listado 3.43 presenta un ejemplo de construcción de la instrucción MOVD.

```
1 MOVD F4, F0 ; F4:F5 <- F0:F1
```

Listado 3.43: MOVD

MOVI2FP / MOVFP2I — MOVI2FP Fd, Rs

Transfiere los bits brutos entre registros enteros y flotantes sin conversión de valor.

Operandos: Fd / Rd: Registro destino — Fd (flotante) en MOVI2FP, Rd (entero) en MOVFP2I. Rs / Fs: Registro fuente — Rs (entero) en MOVI2FP, Fs (flotante) en MOVFP2I. El Listado 3.44 presenta un ejemplo de construcción de la instrucción MOVI2FP.

```
1 MOVI2FP F10, R1 ; F10 <- R1 (bits sin conversion)
```

Listado 3.44: MOVI2FP

Notas: Útil para inicializar registros float con valores enteros antes de una conversión explícita (CVTI2F/CVTI2D).

MOVI2S / MOVS2I — MOVI2S VLR, Rs / MOVS2I Rd, VLR

Transfiere el contenido del registro de longitud vectorial (VLR): MOVI2S carga Rs en VLR; MOVS2I copia VLR a Rd.

Operandos: VLR: Registro de longitud vectorial (destino en MOVI2S, fuente en MOVS2I). Rs: Registro entero fuente (MOVI2S). Rd: Registro entero destino (MOVS2I). El Listado 3.45 presenta un ejemplo de construcción de la instrucción MOVI2S.

```
1 ADDI R1, R0, #16
2 MOVI2S VLR, R1 ; VLR <- 16 (procesar 16 elementos)
3 MOVS2I R2, VLR ; R2 <- VLR actual
```

Listado 3.45: MOVI2S

Notas: MOV12S limita el valor escrito al rango [0, MVL]: si Rs es mayor que MVL, VLR toma el valor de MVL.

MOVF2S / MOVS2F — MOVF2S VM, Fs / MOVS2F Fd, VM

Transfiere el contenido de la máscara vectorial (VM) desde/hacia un registro flotante tratado como *bitmask* de 32 bits.

Operandos: VM: Máscara vectorial (destino en MOVF2S, fuente en MOVS2F). Fs: Registro flotante fuente (MOVF2S). Fd: Registro flotante destino (MOVS2F). El Listado 3.46 presenta un ejemplo de construcción de la instrucción MOVF2S.

1	MOVF2S VM, F0	; VM <- bitmask(bits de F0)
2	MOVS2F F1, VM	; F1 <- bitmask empaquetada de VM

Listado 3.46: MOVF2S

Notas: Solo se representan 32 elementos. VM[32..MVL-1] se ponen a 0 en MOVF2S. Usa CVM para resetear VM con todos los bits a 1.

3.5. Acceso a memoria

En esta sección se listan las operaciones que implican acceso a memoria (cargas y almacenamientos).

LB — LB Rd, LABEL (directo) / LB Rd, offset(Rs) (indirecto)

Carga un byte desde la memoria y extiende el signo a 32 bits.

Directo (2 operandos): la dirección es el símbolo de `.data` indicado, R0 actúa como base implícita.

Indirecto (3 operandos): la dirección efectiva es Rs + desplazamiento (entero decimal o hex, no se permiten etiquetas como desplazamiento).

Operandos: Rd: Registro destino. LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.47 presenta un ejemplo de construcción de la instrucción LB.

1	LB R5, MiByte	; R5 <- MEM[addr(MiByte)]
2	LB R5, 0(R2)	; R5 <- MEM[R2]

Listado 3.47: LB

Notas: No requiere alineación (acceso de 1 byte).

LBU — LBU Rd, LABEL / LBU Rd, offset(Rs)

Carga un byte sin extensión de signo (bits altos a 0).

Operandos: Rd: Registro destino. LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.48 presenta un ejemplo de construcción de la instrucción LBU.

1	LBU R5, MiByte
2	LBU R5, 0(R2)

Listado 3.48: LBU

Notas: No requiere alineación (acceso de 1 byte).

LH — LH Rd, LABEL / LH Rd, offset(Rs)

Carga una media palabra (2 bytes) con extensión de signo.

Operandos: Rd: Registro destino. LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.49 presenta un ejemplo de construcción de la instrucción LH.

1	LH R5, MiMedia
2	LH R5, 2(R2)

Listado 3.49: LH

Notas: Requiere alineación a 2 bytes.

LHU — LHU Rd, LABEL / LHU Rd, offset(Rs)

Carga una media palabra sin extensión de signo (bits altos a 0).

Operandos: Rd: Registro destino. LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.50 presenta un ejemplo de construcción de la instrucción LHU.

1	LHU R5, MiMedia
2	LHU R5, 2(R2)

Listado 3.50: LHU

Notas: Requiere alineación a 2 bytes.

LW — LW Rd, LABEL / LW Rd, offset(Rs)

Carga una palabra de 32 bits desde la memoria a un registro. Soporta direccionamiento directo (etiqueta) e indirecto (offset+base). Exige que la dirección de memoria sea múltiplo de 4.

Operandos: Rd: Registro destino. LABEL: Símbolo de `.data` (modo directo). offset: Entero decimal o hex, sin etiquetas (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.51 presenta un ejemplo de construcción de la instrucción LW.

1	LW R5, MiVar	; R5 <- MEM[addr(MiVar)]
2	LW R5, 4(R2)	; R5 <- MEM[R2+4]

Listado 3.51: LW

Notas: Provoca un riesgo RAW de tipo *Load-Use* si la siguiente instrucción intenta usar el registro Rd inmediatamente. Requiere alineación a 4 bytes.

LF — LF Fd, LABEL / LF Fd, offset(Rs)

Carga un valor de punto flotante de precisión simple (32 bits) en un registro float.

Operandos: Fd: Registro flotante destino (cualquiera F0-F31). LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.52 presenta un ejemplo de construcción de la instrucción LF.

1	LF F0, MiFloat
2	LF F2, 8(R1)

Listado 3.52: LF

Notas: Fd puede ser cualquier registro float (F0-F31). Requiere alineación a 4 bytes.

LD — LD Fd, LABEL / LD Fd, offset(Rs)

Carga un valor de punto flotante de doble precisión (64 bits) en un par de registros (Fn y Fn+1). Requiere alineación de 8 bytes.

Operandos: Fd: Registro flotante destino — debe ser PAR (F0, F2, F4...). LABEL/offset/Rs como en LW. El Listado 3.53 presenta un ejemplo de construcción de la instrucción LD.

1	LD F0, MiDouble
2	LD F0, 0(R1)

Listado 3.53: LD

Notas: El registro destino (Fd) debe ser siempre PAR. El double ocupa Fd (parte alta) y Fd+1 (parte baja). Requiere alineación de 8 bytes.

SB — SB LABEL, Rs2 / SB offset(Rs1), Rs2

Almacena el byte menos significativo de Rs2 en la memoria.

Operandos: LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs1: Registro base (modo indirecto). Rs2: Registro fuente. El Listado 3.54 presenta un ejemplo de construcción de la instrucción SB.

1	SB MiByte, R5
2	SB 0(R2), R5

Listado 3.54: SB

Notas: No requiere alineación (acceso de 1 byte).

SH — SH LABEL, Rs2 / SH offset(Rs1), Rs2

Almacena los 16 bits bajos de Rs2 en la memoria.

Operandos: LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs1: Registro base (modo indirecto). Rs2: Registro fuente. El Listado 3.55 presenta un ejemplo de construcción de la instrucción SH.

1	SH MiMedia, R5
2	SH 2(R2), R5

Listado 3.55: SH

Notas: El simulador no comprueba la alineación en los almacenamientos, pero se recomienda usar direcciones múltiplo de 2 para poder leer el dato después con LH/LHU, que sí la exigen.

SW — SW LABEL, Rs2 / SW offset(Rs1), Rs2

Almacena una palabra (32 bits) en la memoria.

Operandos: LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs1: Registro base (modo indirecto). Rs2: Registro fuente. El Listado 3.56 presenta un ejemplo de construcción de la instrucción SW.

1	SW MiVar, R3	; MEM[addr(MiVar)] <- R3
2	SW 4(R1), R3	; MEM[R1+4] <- R3

Listado 3.56: SW

Notas: El simulador no comprueba la alineación en los almacenamientos, pero se recomienda usar direcciones múltiplo de 4 para poder leer el dato después con LW, que sí la exige.

SF — **SF LABEL, Fs / SF offset(Rs), Fs**

Almacena un float de precisión simple desde un registro flotante.

Operandos: LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico (modo indirecto). Rs: Registro base (modo indirecto). Fs: Registro flotante fuente (cualquiera F0-F31). El Listado 3.57 presenta un ejemplo de construcción de la instrucción **SF**.

```
1 SF MiFloat, F2
2 SF 8(R1), F2
```

Listado 3.57: SF

Notas: El simulador no comprueba la alineación en los almacenamientos, pero se recomienda usar direcciones múltiplo de 4 para poder leer el dato después con **LF**, que sí la exige.

SD — **SD LABEL, Fs / SD offset(Rs), Fs**

Almacena un valor de 64 bits (Double) de los registros Fs:F_s+1 en la memoria.

Operandos: LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico (modo indirecto). Rs: Registro base (modo indirecto). Fs: Registro flotante fuente, debe ser PAR (F0, F2, F4...). El Listado 3.58 presenta un ejemplo de construcción de la instrucción **SD**.

```
1 SD MiDouble, F0
2 SD 0(R1), F0
```

Listado 3.58: SD

Notas: Fs debe ser un registro par. El simulador no comprueba la alineación en los almacenamientos, lo que permite, por ejemplo, guardar un double en un bloque de parámetros de TRAP 5 alineado a 4 bytes. Si el dato se va a leer después con **LD**, la dirección debe ser múltiplo de 8, ya que **LD** sí exige esa alineación.

3.6. Saltos y bifurcaciones

En esta sección se listan las operaciones de salto condicionales (bifurcaciones) e incondicionales.

BEQZ — **BEQZ Rs, label**

Salta a la etiqueta si Rs es igual a 0 (Branch if Equal to Zero).

Operandos: Rs: Registro a comparar con 0. label: etiqueta destino, solo de `.text` (no se permiten etiquetas de `.data`). El Listado 3.59 presenta un ejemplo de construcción de la instrucción **BEQZ**.

```
1 BEQZ R1, fin      ; si R1 == 0, salta a la etiqueta "fin"
```

Listado 3.59: BEQZ

Notas: La etiqueta debe estar definida en la sección `.text`. El ensamblador la resuelve a la dirección absoluta de la instrucción destino, codificada como número de palabra (dirección / 4) en el campo de 16 bits; no es un desplazamiento relativo al PC. Lo mismo se aplica al resto de saltos condicionales.

BNEZ — BNEZ Rs, label

Salta si Rs es distinto de 0 (Branch if Not Equal to Zero).

Operandos: Rs: Registro a comparar con 0. label: Etiqueta destino, solo de `.text`. El Listado 3.60 presenta un ejemplo de construcción de la instrucción BNEZ.

```
1 BNEZ R1, bucle
```

Listado 3.60: BNEZ

Notas: Solo se permiten etiquetas de la sección de código (`.text`).

BGTZ — BGTZ Rs, label

Salta si Rs es mayor que 0, con signo (Branch if Greater Than Zero).

Operandos: Rs: Registro a comparar con 0. label: Etiqueta destino, solo de `.text`. El Listado 3.61 presenta un ejemplo de construcción de la instrucción BGTZ.

```
1 BGTZ R2, positivo
```

Listado 3.61: BGTZ

Notas: Solo se permiten etiquetas de `.text`.

BLTZ — BLTZ Rs, label

Salta si Rs es menor que 0, con signo (Branch if Less Than Zero).

Operandos: Rs: Registro a comparar con 0. label: Etiqueta destino, solo de `.text`. El Listado 3.62 presenta un ejemplo de construcción de la instrucción BLTZ.

```
1 BLTZ R2, negativo
```

Listado 3.62: BLTZ

Notas: Solo se permiten etiquetas de `.text`.

BFPT — BFPT label

Salta si el bit de condición de punto flotante (`fpConditionBit`) es 1. El bit lo fijan las instrucciones de comparación float (EQF, LTD, etc.).

Operandos: label: Etiqueta destino, solo de `.text`. El Listado 3.63 presenta un ejemplo de construcción de la instrucción BFPT.

```
1 LED F0, F2
2 BFPT menor_igual
```

Listado 3.63: BFPT

Notas: Solo se permiten etiquetas de código (`.text`). El bit de condición es compartido con BFPPF.

BFPPF — BFPPF label

Salta si el bit de condición de punto flotante es 0. El bit lo fijan las instrucciones de comparación float (EQF, LTD, etc.).

Operandos: label: Etiqueta destino, solo de `.text`. El Listado 3.64 presenta un ejemplo de construcción de la instrucción BFPPF.

```
1 EQF F0, F2
2 BFPPF no_igual
```

Listado 3.64: BFPPF

Notas: Solo se permiten etiquetas de `.text`.

J — J label

Salto incondicional. El PC se actualiza a la dirección de la etiqueta.

Operandos: label: Etiqueta destino, solo de `.text`. El Listado 3.65 presenta un ejemplo de construcción de la instrucción J.

```
1 J bucle
```

Listado 3.65: J

Notas: Solo se permiten etiquetas de código (`.text`). El destino se codifica como dirección absoluta de palabra (dirección / 4) en el campo de 26 bits; no es un desplazamiento relativo al PC. Lo mismo se aplica a JAL.

JAL — JAL label

Salto con enlace (Jump and Link). Guarda la dirección de retorno (PC+4) en R31 y salta a la etiqueta.

Operandos: label: Etiqueta destino, solo de `.text`. El Listado 3.66 presenta un ejemplo de construcción de la instrucción JAL.

```
1 JAL miFuncion ; R31 = PC+4; PC = miFuncion
```

Listado 3.66: JAL

Notas: R31 es el registro de retorno por convención en ASG. Solo se permiten etiquetas de código (`.text`).

JR — JR Rs

Salto incondicional a la dirección contenida en el registro Rs.

Operandos: Rs: Registro entero con la dirección destino. El Listado 3.67 presenta un ejemplo de construcción de la instrucción JR.

```
1 JR R31 ; retorno de funcion
```

Listado 3.67: JR

Notas: No usa etiqueta: la dirección se toma del registro. Se usa típicamente para el retorno de subrutinas (JR R31).

JALR — JALR Rs

Salto con enlace por registro. Guarda PC+4 en R31 y salta a la dirección contenida en Rs.

Operandos: Rs: Registro entero con la dirección destino. El Listado 3.68 presenta un ejemplo de construcción de la instrucción JALR.

```
1 JALR R2 ; R31 <- PC+4, salta a la direccion de R2
```

Listado 3.68: JALR

Notas: No usa etiqueta: la dirección destino viene del registro Rs. El enlace se guarda siempre en R31 (igual que en JAL); no se puede indicar otro registro destino.

3.7. Instrucciones vectoriales

En esta sección se listan las diferentes operaciones vectoriales implementadas en el sistema: aritméticas, comparaciones, accesos a memoria y operaciones especiales.

ADDV / SUBV / MULTV / DIVV — ADDV Vd, Vs1, Vs2

Aritmética vector-vector, elemento a elemento, sobre los VLR elementos activos.

Operandos: Vd: Registro vectorial destino. Vs1: Primer vector fuente. Vs2: Segundo vector fuente. El Listado 3.69 presenta un ejemplo de construcción de la instrucción ADDV.

```
1  ADDV V0, V1, V2 ; V0[i] = V1[i] + V2[i]
```

Listado 3.69: ADDV

Notas: Con el encadenamiento vectorial (chaining) activado, una instrucción que depende del resultado de otra (dependencia RAW) puede empezar a consumir sus elementos a medida que se producen, sin esperar al vector completo. Latencias por defecto: ADDV/SUBV 6 ciclos, MULTV 7 ciclos y DIVV 20 ciclos (las mismas para las variantes escalar-vector y vector-escalar).

ADDSV / SUBSV / MULTSV / DIVSV — ADDSV Vd, Fs, Vs

Aritmética escalar-vector: el escalar (registro flotante Fs) opera con cada elemento del vector Vs.

Operandos: Vd: Registro vectorial destino. Fs: Fuente escalar flotante. Vs: Vector fuente. El Listado 3.70 presenta un ejemplo de construcción de la instrucción MULTSV.

```
1  MULTSV V0, F2, V1 ; V0[i] = F2 * V1[i]
```

Listado 3.70: MULTSV

ADDVS / SUBVS / MULTVS / DIVVS — ADDVS Vd, Vs, Fs

Aritmética vector-escalar: cada elemento del vector Vs opera con el escalar Fs.

Operandos: Vd: Registro vectorial destino. Vs: Vector fuente. Fs: Fuente escalar flotante. El Listado 3.71 presenta un ejemplo de construcción de la instrucción ADDVS.

```
1  ADDVS V0, V1, F2 ; V0[i] = V1[i] + F2
```

Listado 3.71: ADDVS

SEQV / SNEV / SGTV / SLTV / SGEV / SLEV — SEQV Vs1, Vs2

Comparación vector-vector; el resultado elemento a elemento se escribe directamente en VM: $VM[i] = \text{condición}(Vs1[i], Vs2[i])$.

Operandos: Vs1: Primer vector fuente. Vs2: Segundo vector fuente. El Listado 3.72 presenta un ejemplo de construcción de la instrucción SLTV.

```
1  SLTV V1, V2      ; VM[i] = (V1[i] < V2[i])
```

Listado 3.72: SLTV

Notas: No tiene registro destino explícito; el resultado siempre va a VM. Usa CVM para resetear la máscara. Latencia por defecto de todas las comparaciones vectoriales: 1 ciclo.

SEQSV / SNESV / SGTSV / SLTSV / SGEV / SLESV — SEQSV Fs, Vs

Comparación escalar-vector: $VM[i] = \text{condición}(Fs, Vs[i])$.

Operandos: Fs: Fuente escalar flotante. Vs: Vector fuente. El Listado 3.73 presenta un ejemplo de construcción de la instrucción SLTSV.

```
1  SLTSV F2, V1     ; VM[i] = (F2 < V1[i])
```

Listado 3.73: SLTSV

Notas: El resultado se escribe en VM, no en un vector destino.

SEQVS / SNEVS / SGTVS / SLTVS / SGEVS / SLEVS — SEQVS Vs, Fs

Comparación vector-escalar: $VM[i] = \text{condición}(Vs[i], Fs)$.

Operandos: Vs: Vector fuente. Fs: Fuente escalar flotante. El Listado 3.74 presenta un ejemplo de construcción de la instrucción SGEVS.

```
1  SGEVS V1, F2     ; VM[i] = (V1[i] >= F2)
```

Listado 3.74: SGEVS

Notas: El resultado se escribe en VM, no en un vector destino.

LV / SV — LV Vd, LABEL / LV Vd, offset(Rs) / SV LABEL, Vs / SV offset(Rs), Vs

Carga (LV) o almacena (SV) un vector, elemento a elemento, en posiciones consecutivas de memoria a partir de la dirección base. El número de elementos que se transfieren lo controla VLR.

Operandos: Vd: Registro vectorial destino (LV). Vs: Registro vectorial fuente (SV). LABEL: Símbolo de `.data` (modo directo). offset: Desplazamiento numérico con signo (modo indirecto). Rs: Registro base (modo indirecto). El Listado 3.75 presenta un ejemplo de construcción de las instrucciones LV y SV.

```
1  LV V0, vector_x ; carga VLR elementos desde la etiqueta vector_x
2  LV V1, 0(R1)    ; carga VLR elementos desde MEM[R1]
3  SV 0(R2), V1    ; guarda VLR elementos en MEM[R2]
```

Listado 3.75: LV y SV

Notas: Cada elemento ocupa 8 bytes (double), por lo que el elemento i se lee o escribe en la dirección $\text{base} + 8 \times i$. Los datos se declaran con `.double`. La dirección base debe estar alineada a 8 bytes: si no lo está, LV lanza una excepción de dirección desalineada. Con solo dos operandos, el segundo se interpreta siempre como etiqueta de `.data`, por lo que LV V0, R1 no es válido: para usar un registro como base hay que escribir LV V0, 0(R1). Latencia por defecto de todos los accesos a memoria vectorial (LV, SV, LVWS, SVWS, LVI, SVI): 12 ciclos.

LVWS / SVWS — LVWS Vd, Rs1, Rs2

Carga/almacena con paso (*stride*): Rs2 contiene el paso en bytes entre elementos consecutivos.

Operandos: Vd: Registro vectorial destino. Rs1: Registro entero con dirección base. Rs2: Registro entero con el stride (salto entre elementos). El Listado 3.76 presenta un ejemplo de construcción de la instrucción LVWS.

```
1  LVWS V0, R1, R2 ; V0[i] = MEM[R1 + i*R2]
```

Listado 3.76: LVWS

LVI / SVI — LVI Vd, Rs, Vi

Carga/almacena indexado: cada elemento usa su propio desplazamiento tomado del vector de índices Vi.

Operandos: Vd: Registro vectorial destino. Rs: Registro entero con dirección base. Vi: Vector de índices; cada elemento aporta su propio desplazamiento en bytes respecto a la dirección base. El Listado 3.77 presenta un ejemplo de construcción de la instrucción LVI.

```
1  LVI V0, R1, V1 ; V0[i] = MEM[R1 + V1[i]]
```

Listado 3.77: LVI

Notas: Como cada elemento ocupa 8 bytes, los índices deben ser múltiplos de 8 para que las direcciones resultantes estén alineadas.

CVI — CVI Vd, Rs

Crea un vector de índices escalados en Vd: cada elemento i recibe $i \times Rs$.

Operandos: Vd: Registro vectorial destino. Rs: Registro entero con el factor de escala. El Listado 3.78 presenta un ejemplo de construcción de la instrucción CVI.

```
1  ADDI R1, R0, #8
2  CVI V0, R1      ; V0 = [0, 8, 16, 24, ...] (V0[i] <- i*8)
```

Listado 3.78: CVI

Notas: Con Rs=8 genera la secuencia 0, 8, 16, 24..., que sirve como vector de índices para accesos indirectos (LVI/SVI) a elementos consecutivos, ya que los índices son desplazamientos en bytes y cada elemento ocupa 8 bytes. Con Rs=16 se accedería a uno de cada dos elementos, y así sucesivamente.

CVM — CVM

Reinicia la máscara vectorial con todos los bits a 1 (todos los elementos activos). El Listado 3.79 presenta un ejemplo de construcción de la instrucción CVM.

```
1  CVM
```

Listado 3.79: CVM

POP — POP Rd, VM

Recuento de población (*popcount*) de la máscara vectorial: cuenta bits a 1 en VM y guarda el resultado en Rd.

Operandos: Rd: Registro destino. VM: Máscara vectorial cuyos bits a 1 se cuentan. El Listado 3.80 presenta un ejemplo de construcción de la instrucción POP.

```
1  POP R1, VM      ; R1 <- numero de elementos activos en VM
```

Listado 3.80: POP

Notas: Útil para saber cuántos elementos de un vector cumplen una condición tras una comparación vectorial (SLTV, SEQV, etc.).

3.8. Directivas

En esta sección se listan las directivas disponibles del ensamblador.

.data — .data [dirección]

Marca el inicio del segmento de datos. Las directivas siguientes (`.word`, `.space`, `.asciiz`, etc.) reservan espacio en la memoria de datos. Opcionalmente admite una dirección inicial: los datos siguientes se colocan a partir de esa dirección de la memoria de datos. El Listado 3.81 presenta un ejemplo de uso de la directiva `.data`.

```
1  .data
2  MiVar: .word 42
3  .data 0x1000      ; los datos siguientes empiezan en 0x1000
4  X:      .double 1.0, 2.0
```

Listado 3.81: `.data`**.text — .text [desplazamiento]**

Marca el inicio del segmento de código. Las instrucciones siguientes se ensamblan en la memoria de instrucciones. Es obligatoria: todo programa debe contener al menos una directiva `.text`. Opcionalmente admite un desplazamiento, relativo al inicio del código del usuario (0x64, justo después de la tabla de vectores). El Listado 3.82 presenta un ejemplo de uso de la directiva `.text`.

```
1  .text
2  ADDI R1, R0, #5
```

Listado 3.82: `.text`**.global — .global label**

Declara una etiqueta como global, visible para todos los módulos que se ensamblen juntos. Sin esta directiva, las etiquetas son locales al módulo. El Listado 3.83 presenta un ejemplo de uso de la directiva `.global`.

```
1  .global InputUnsigned
2  InputUnsigned:
```

Listado 3.83: `.global`

Notas: Necesaria cuando se ensamblan varios archivos (`.s`) juntos y un módulo referencia etiquetas de otro.

.word — .word value [, value2, ...]

Reserva una o más palabras de 32 bits en la memoria de datos y las inicializa con los valores dados. Se puede usar con etiquetas para referenciar la dirección. El Listado 3.84 presenta un ejemplo de uso de la directiva `.word`.

```
1  MiArray: .word 1, 2, 3, 4
```

Listado 3.84: `.word`

`.byte` — `.byte value [, value2, ...]`

Reserva uno o más bytes en la memoria de datos. El Listado 3.85 presenta un ejemplo de uso de la directiva `.byte`.

```
1  MiByte: .byte 0xFF
```

Listado 3.85: `.byte`

`.float` — `.float value [, value2, ...]`

Reserva 4 bytes por cada valor y los inicializa con valores de punto flotante de precisión simple. El Listado 3.86 presenta un ejemplo de uso de la directiva `.float`.

```
1  Pi: .float 3.14159
```

Listado 3.86: `.float`

`.double` — `.double value [, value2, ...]`

Reserva 8 bytes por cada valor y los inicializa con valores de punto flotante de doble precisión. Es el formato de los elementos de los registros vectoriales (LV/SV). El Listado 3.87 presenta un ejemplo de uso de la directiva `.double`.

```
1  E: .double 2.718281828
```

Listado 3.87: `.double`

`.space` — `.space n`

Reserva `n` bytes en la memoria de datos. Su contenido inicial es 0, ya que la memoria de datos se pone a cero antes de cargar cada programa. El Listado 3.88 presenta un ejemplo de uso de la directiva `.space`.

```
1  Buffer: .space 80
```

Listado 3.88: `.space`

.ascii — .ascii "string"

Almacena la cadena de texto en memoria sin terminador nulo. Se procesan las secuencias de escape estándar (`\n`, `\t`, `\r`, `\\`, `\"`, `\0`). El Listado 3.89 presenta un ejemplo de uso de la directiva `.ascii`.

```
1 .ascii "Hola mundo"
```

Listado 3.89: `.ascii`**.asciiz — .asciiz "string"**

Almacena la cadena de texto seguida de un byte nulo (0x00). Es la forma estándar para cadenas compatibles con C. El Listado 3.90 presenta un ejemplo de uso de la directiva `.asciiz`.

```
1 Msg: .asciiz "Valor = %d\n"
```

Listado 3.90: `.asciiz`

Notas: Soporta secuencias de escape: `\n` (0x0A), `\t` (0x09), `\r` (0x0D), `\\` (0x5C), `\"` (0x22), `\0` (0x00).

.align — .align n

Alinea el puntero del segmento actual (datos o código) al siguiente múltiplo de 2^n bytes. Útil antes de `.word` o `.double` para cumplir requisitos de alineación. El Listado 3.91 presenta un ejemplo de uso de la directiva `.align`.

```
1 .align 2 ; alinea a 4 bytes (2^2)
2 .word 0
```

Listado 3.91: `.align`

3.9. Llamadas al sistema

En esta sección se listan las operaciones especiales del sistema, el manejo de excepciones y el control de entrada/salida.

IAR — registro especial implícito

El IAR (Interrupt Address Register) es un registro especial de 32 bits que escribe automáticamente el procesador. Al producirse una excepción guarda la dirección de la instrucción que la ha

provocado, y es la dirección que utiliza RFE para continuar la ejecución. Al ejecutar un TRAP guarda la dirección de la instrucción siguiente al TRAP (PC+4). El Listado 3.92 presenta un ejemplo de uso del registro especial IAR.

```
1  DIV R3, R1, R2    ; si R2 = 0: IAR <- PC de esta DIV
2                      ; y salta al manejador __divzero_handler
```

Listado 3.92: IAR

Notas: El IAR se muestra en el panel de registros (banco de enteros, al final). Ninguna instrucción de usuario puede escribirlo. Se reinicia a 0 cuando el procesador se reinicia.

RFE — RFE

Retorna de un manejador de excepciones. Carga en el PC la dirección guardada en el IAR más 4 ($PC \leftarrow IAR + 4$), borra el registro de causa de la excepción y vacía las instrucciones captadas especulativamente tras RFE. Como el IAR contiene la dirección de la instrucción que provocó la excepción, la ejecución continúa en la instrucción siguiente a ella. El Listado 3.93 presenta un ejemplo de construcción de la instrucción RFE.

```
1  __divzero_handler: ; se llega aquí si una DIV divide por 0
2  ADDI R3, R0, #0    ; tratamiento: deja el resultado a 0
3  RFE                ; PC <- IAR+4: continua tras la DIV
```

Listado 3.93: RFE

Notas: RFE es una instrucción de tipo J sin operandos. La instrucción que provocó la excepción no se vuelve a ejecutar. Los TRAP no pasan por ningún manejador (el simulador los ejecuta directamente), por lo que no es necesario usar RFE tras un TRAP.

TRAP 0 — exit

Finaliza la ejecución del programa. El simulador detiene el pipeline y marca la ejecución como terminada. El Listado 3.94 presenta un ejemplo de construcción de la instrucción TRAP 0.

```
1  TRAP 0
```

Listado 3.94: TRAP 0

TRAP 1 — open

Apertura de fichero (no soportada en este simulador). Devuelve siempre -1 en R1. El Listado 3.95 presenta un ejemplo de construcción de la instrucción TRAP 1.

```
1  TRAP 1
```

Listado 3.95: TRAP 1

Notas: No implementada. Devuelve -1 para indicar error.

TRAP 2 — close

Cierre de fichero (no soportado en este simulador). Devuelve siempre -1 en R1. El Listado 3.96 presenta un ejemplo de construcción de la instrucción **TRAP 2**.

```
1 TRAP 2
```

Listado 3.96: TRAP 2**TRAP 3 — read**

Lee una línea de texto desde la entrada estándar (stdin). R14 debe apuntar a un bloque de parámetros: MEM[R14+0] (word) descriptor de fichero (0=stdin); MEM[R14+4] (word) dirección del buffer destino; MEM[R14+8] (word) tamaño del buffer en bytes. La línea leída incluye el salto de línea final y se termina con un byte nulo, por lo que se guardan como máximo (tamaño $- 1$) caracteres; si la línea es más larga, se trunca. Al terminar, R1 contiene el número de caracteres guardados (sin contar el nulo). Solo se admite el descriptor 0 (stdin); con cualquier otro valor, R1 recibe -1 .

Operandos: R14: Dirección del bloque de parámetros en memoria de datos. El Listado 3.97 presenta un ejemplo de construcción de la instrucción **TRAP 3**.

```
1 .data
2 LeerBuf: .space 80
3 LeerPar: .word 0, LeerBuf, 80
4 .text
5 ADDI R14, R0, LeerPar
6 TRAP 3
```

Listado 3.97: TRAP 3

Notas: La ejecución se pausa hasta que el usuario introduce texto en la consola. El simulador muestra un aviso de entrada en la propia consola.

TRAP 4 — write

Escribe bytes en stdout o stderr. Bloque de parámetros: MEM[R14+0] descriptor (1=stdout, 2=stderr); MEM[R14+4] dirección del buffer origen; MEM[R14+8] número de bytes a escribir. Escribe exactamente ese número de bytes, sin detenerse en un byte nulo, por lo que en una

cadena `.asciiz` hay que indicar su longitud sin contar el nulo. Al terminar, R1 contiene el número de bytes escritos (−1 si el descriptor no es válido).

Operandos: R14: Dirección del bloque de parámetros en memoria de datos. El Listado 3.98 presenta un ejemplo de construcción de la instrucción TRAP 4.

```
1  .data
2  Msg:  .asciiz "Hola"
3  .align 2
4  EscrPar: .word 1, Msg, 4    ; 4 = longitud de "Hola"
5  .text
6  ADDI R14, R0, EscrPar
7  TRAP 4
```

Listado 3.98: TRAP 4

TRAP 5 — printf

Imprime en stdout con formato estilo printf. Bloque de parámetros: MEM[R14+0] dirección de la cadena de formato (`.asciiz`); MEM[R14+4], MEM[R14+8]... argumentos según especificador (`%d/%i` entero con signo, `%u` sin signo, `%x/%X` hexadecimal, `%f/%g/%e` double, `%s` cadena, `%c` carácter y `%%` para escribir el símbolo %). Al terminar, R1 contiene el número de caracteres escritos.

Operandos: R14: Dirección del bloque de parámetros en memoria de datos. El Listado 3.99 presenta un ejemplo de construcción de la instrucción TRAP 5.

```
1  .data
2  Fmt:  .asciiz "Factorial = %g"
3  .align 2
4  Par:  .word Fmt
5  Val:  .space 8
6  .text
7  SD Val, F2
8  ADDI R14, R0, Par
9  TRAP 5
```

Listado 3.99: TRAP 5

Notas: Los argumentos float/double (`%f,%g,%e`) deben guardarse como double (8 bytes, alineados a 4). Los enteros como word (4 bytes). Las cadenas como dirección word.

TRAP 6 — parada inmediata

Detiene inmediatamente la ejecución del programa, descartando las instrucciones posteriores y sin modificar R1 ni ningún otro registro, de forma que el estado de la máquina queda tal y como estaba al llegar al TRAP. Es la instrucción que contiene el manejador por defecto (`__default_handler`, en la dirección 0x60), al que saltan todas las excepciones para las que

el usuario no ha definido un manejador propio. El Listado 3.100 presenta un ejemplo de construcción de la instrucción TRAP 6.

1	TRAP 6 ; detiene el procesador de inmediato
---	---

Listado 3.100: TRAP 6

Notas: A diferencia de TRAP 0, que finaliza el programa de forma ordenada, TRAP 6 es una parada de emergencia. No escribe nada en la consola: el motivo de la parada se muestra en el “Log de eventos”. Los códigos de TRAP válidos son del 0 al 6.